SISTEMA DE GESTIÓN EMPRESARIAL

UD06 –Desarrollo de módulos de Odoo Modelo y Vista

Actividades Evaluables 01



Miriam Mendieta Racedo.-
2DAMS
22/12/2025

ÍNDICE

INTRODUCCIÓN **1**

ACTIVIDAD 01 2
ACTIVIDAD 02 11
ACTIVIDAD 03 27
ACTIVIDAD 04 38

BIBLIOGRAFIA **54**

INTRODUCCIÓN

Odoo opera bajo una arquitectura modular , lo que lo convierte en un Sistema de Gestión Empresarial (ERP) flexible y potente. Está compuesto por una vasta colección de módulos (o aplicaciones) que se pueden instalar y combinar según necesidades de cada empresa.

Cada módulo en Odoo cumple una función empresarial específica:

- Modelos de Datos (Models): Un módulo define sus propios modelos de datos , que son la estructura de la información que gestiona.

- Relaciones: Los módulos establecen relaciones entre estos modelos , permitiendo que la información fluya entre diferentes partes del sistema.

- Vistas (Views): Definen cómo se presenta la información al usuario (formularios, listas, o formatos específicos como Kanban o Calendar).

- Lógica de Negocio (Business Logic): Contienen el código Python que implementa las reglas y los flujos de trabajo.

La verdadera fortaleza de Odoo reside en su capacidad de personalización a través de estos módulos. La personalización permite adaptar el ERP a los procesos únicos de cualquier negocio , lo cual es el foco principal de estas actividades evaluables:

- Creación de Nuevos Modelos y Módulos: implica definir completamente nuevos modelos de datos y sus interrelaciones.
- Ampliación de Funcionalidad Existente: Se puede tomar un módulo base y ampliarlo para añadir nuevos campos y modelos relacionados .
- Diseño de Vistas Específicas: Odoo permite crear vistas altamente funcionales que facilitan la toma de decisiones, como las vistas Kanban y Calendar por ejemplo.
- Implementación de Reglas de Seguridad: Es posible definir roles de usuario y establecer reglas de acceso detalladas (quién puede ver, quién puede modificar) a nivel de modelo.

En resumen, la arquitectura modular de Odoo ofrece al desarrollador las herramientas para extender y redefinir casi cualquier aspecto del sistema, asegurando que el ERP se ajuste perfectamente a la realidad operativa de la empresa.

ACTIVIDAD 01

Modifica el ejemplo más sencillo de la lista de tareas de forma que:

- Tenga una nueva vista para ver las tareas en formato “Kanban”.
- Modifica las tareas para tener una fecha asignada y crea una nueva vista para ver en una vista “Calendar” la fecha asignada.

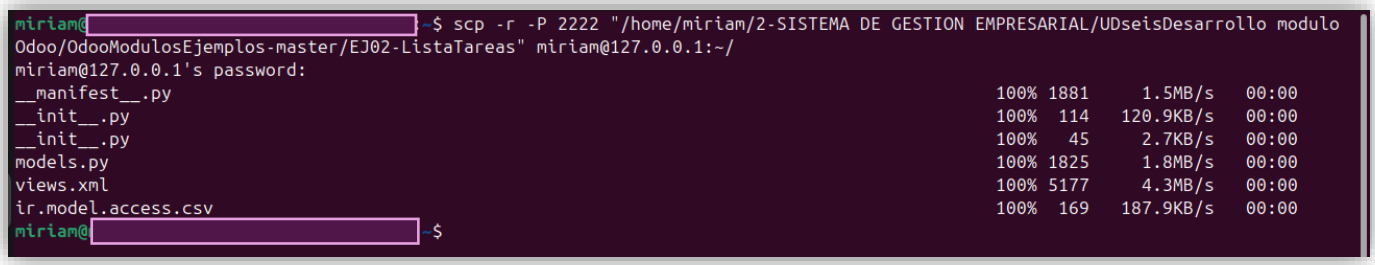
Proceso de Transferencia de Módulo Odoo: Lista de Tareas

Para transferir la carpeta del módulo Odoo (ListaTareas) desde el ordenador anfitrión a la carpeta de volúmenes de addons de la VM Ubuntu Server, asegurando la conectividad y permisos correctos.

- Módulo de Origen: Carpeta Ejemplo_ListaTareas .
- Destino de Odoo (VM): ~/empresa/desarrollo/volumesOdoo/addons/
- Conexión SSH: Se utiliza el reenvío de puertos con la IP local (127.0.0.1) y el puerto 2222
- Desde el terminal del ordenador anfitrión se ejecuta el siguiente comando:

  scp -r -P 2222 "/ruta/absoluta/a/Ejemplo_ListaTareas" miriam@127.0.0.1:~/

  La función de este código es copiar la carpeta de origen (-r) al directorio personal (:~/) del usuario miriam en la VM, usando el puerto 2222 (-P 2222).



- Se inicia la sesión SSH en la VM (ssh miriam@127.0.0.1 -p 2222) para realizar la gestión de archivos. (ls ~ Verificación: Confirma que la carpeta ListaTareas se copió al directorio personal.)
- Mover la carpeta del módulo desde el directorio personal a la ubicación donde Odoo la buscará. (mv ~/Ejemplo_ListaTareas ~/empresa/desarrollo/volumesOdoo/addons/

- Navegar al directorio donde se encuentra el nuevo módulo.(cd ~/empresa/desarrollo/volumesOdoo/addons/

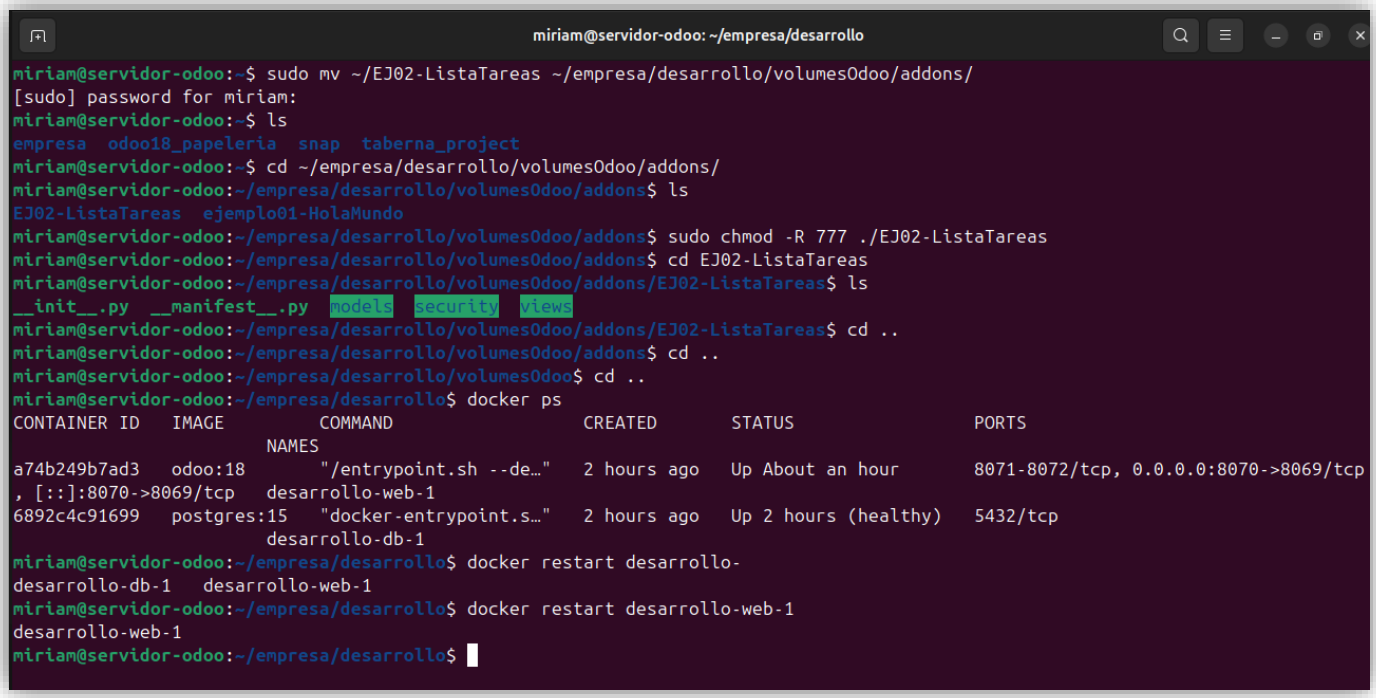
- Asignación de Permisos: Otorga permisos chmod -R 777 (lectura, escritura y ejecución para todos) de forma recursiva (-R) a la carpeta del módulo(sudo chmod -R 777 .)



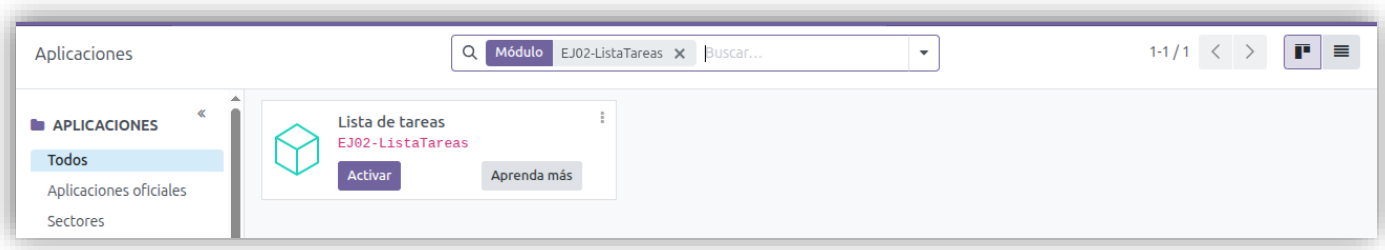
- Reiniciar el Contenedor Odoo: se debe estar en el mismo directorio del archivo docker-compose.yml

sudo docker compose restart nombre/ID de tu contenedor Odoo

Para saber el nombre/ID de tu contenedor Odoo → sudo docker ps (desarrollo-web-1)



- Activar el Modo Desarrollador para actualizar la lista de módulos:
    - En la web de Odoo iniciar sesión como un usuario con derechos de administrador (http://localhost:8070)
    - Se debe tener activado un módulo por ejemplos ventas.
    - Ir al menú principal y luego a la aplicación Ajustes (Settings).
    - Desplázarse hacia abajo hasta el final de la página y haz clic en "Activar el modo desarrollador" (o "Activate the developer mode").
    - Aparece un pequeño ícono de un insecto en la barra superior, lo que confirma que se está en modo desarrollador.
    - Con el modo desarrollador activado ir a aplicaciones y aparecen nuevos botones clicar en un botón llamado "Actualizar lista de Módulos" (o "Update Apps List"), para forzar el escaneo de la carpeta addons en busca de nuevos módulos.
    - En la barra de búsqueda escribir el nombre del módulo (Ejemplo_ListaTareas), si el módulo aparece ciclar en instalar, con esto, el módulo estará funcional en Odoo 18.



Nueva vista para ver las tareas en formato "Kanban"

El objetivo de la vista Kanban es mostrar las tareas como "tarjetas" y agruparlas por columnas. Para agregar una vista Kanban al módulo lista_tareas en Odoo implica modificar los archivos XML de las vistas.(views.xml).

- Navegar hacia la carpeta views para localizar el archivo XML :

  cd empresa1/desarrollo/volumesOdoo/addons/Ejemplo_ListaTareas/views
  nano views.xml

- Insertar el Código de la Vista Kanban:
  1. Modificar la Acción de Ventana (action_lista_tareas): es decirle a Odoo que la vista Kanban debe ser una opción disponible cuando el usuario entra al módulo.

```
<record model="ir.actions.act_window" id="action_lista_tareas">
    <field name="res_model">lista_tareas.lista</field>
    <field name="view_mode">list,form,kanban</field>
</record>
```

  2. Añadir el código completo de la nueva vista:

```
<record id="view_lista_tareas_kanban" model="ir.ui.view">
    <field name="name">lista.tareas.kanban</field>
    <field name="model">lista_tareas.lista</field>
    <field name="arch" type="xml">
     <kanban default_group_by="realizada">

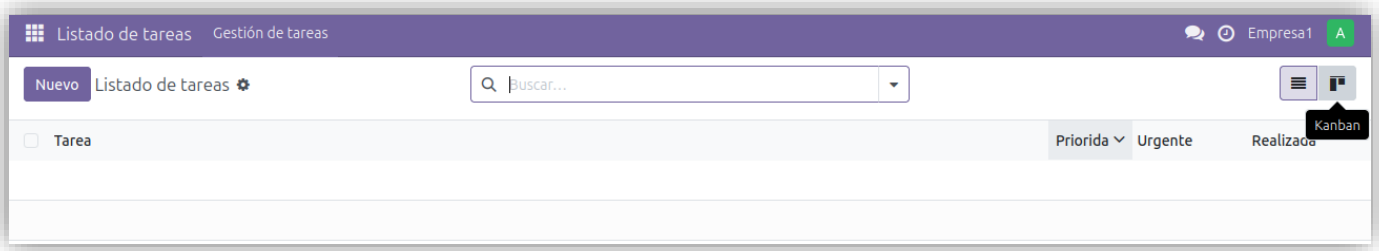
       <field name="tarea"/>
       <field name="prioridad"/>
       <field name="realizada"/>
       <field name="urgente"/>

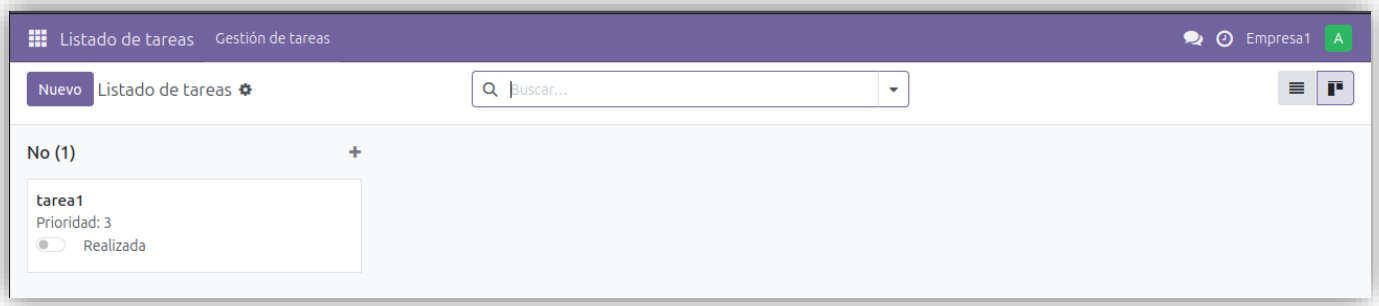
       <templates>
        <t t-name="kanban-box">
         <div t-attf-class="oe_kanban_global_click">
          <div class="o_kanban_record_body">
           <div class="o_kanban_record_title">
            <field name="tarea"/>
           </div>

           <div class="o_kanban_tags">
            Prioridad: <field name="prioridad"/>
           </div>
```

```
            <div class="o_kanban_record_bottom">
              <div class="oe_kanban_bottom_right">
               <field name="realizada" widget="boolean_toggle"/>
              </div>
             </div>
            </div>
           </div>
          </t>
        </templates>
      </kanban>
    </field>
</record>
```

3. Guardar, Reiniciar y Actualizar
   - ✓ Guardar el archivo views.xml con los dos cambios anteriores(Ctrl+O, Enter, Ctrl+X)
   - ✓ Reiniciar el contenedor (desde la ubicación de tu docker-compose.yml):
     docker compose restart desarrollo-web-1
     docker start desarrollo-web-1
   - ✓ Actualiza el Módulo en Odoo (Navegador):
     - Entrar a Odoo.
     - ir a Aplicaciones (Apps) y tener el modo desarrollador activado.
     - Busca tu módulo Listado de tareas y tres puntitos clic en Actualizar.
   - ✓ Al entrar a tu módulo, y estará el icono de la vista Kanban disponible en la esquina superior derecha.





Fecha asignada y vista "Calendar"

1. Modificar el Modelo Python (models.py) para incluir el campo de fecha.

```
# -*- coding: utf-8 -*-
from odoo import models, fields, api

class ListaTareas(models.Model):
    _name = 'lista_tareas.lista'
    _description = 'Modelo de la lista de tareas'
    _rec_name ="tareas"

    tarea = fields.Char(string="Tarea")
    prioridad = fields.Integer(string="Prioridad")
    urgente = fields.Boolean(string="Urgente, compute="_value_urgente", store=True)
    realizada = fields.Boolean(string="Realizada")

    # campo de fecha y hora para la vista Calendar
    date_assigned = fields.Datetime(string='Fecha y Hora Asignada')

    #este método se ejecuta cada vez que cambie el campo "prioridad"
    @api.depends('prioridad')
    # Funcion para calcular el valor de urgente
    def _value_urgente(self):
        for record in self:
            if record.prioridad > 10:
```

2. Modificar la Vista de Formulario XML (views.xml) para que ese campo sea editable. Debes incluir el nuevo campo date_assigned en el Formulario para que el usuario pueda establecer la fecha al crear o editar una tarea.

views.xml

```
<record id="view_lista_tareas_form" model="ir.ui.view">
  <field name="name">lista.tareas.form</field>
  <field name="model">lista_tareas.lista</field>
  <field name="arch" type="xml">
    <form string="Tarea">
      <sheet>
        <group>
          <field name="tarea"/>
          <field name="realizada"/>
        </group>

        <group string="Prioridad y urgencia">
          <field name="prioridad"/>
          <field name="urgente" readonly="1"/>
        </group>

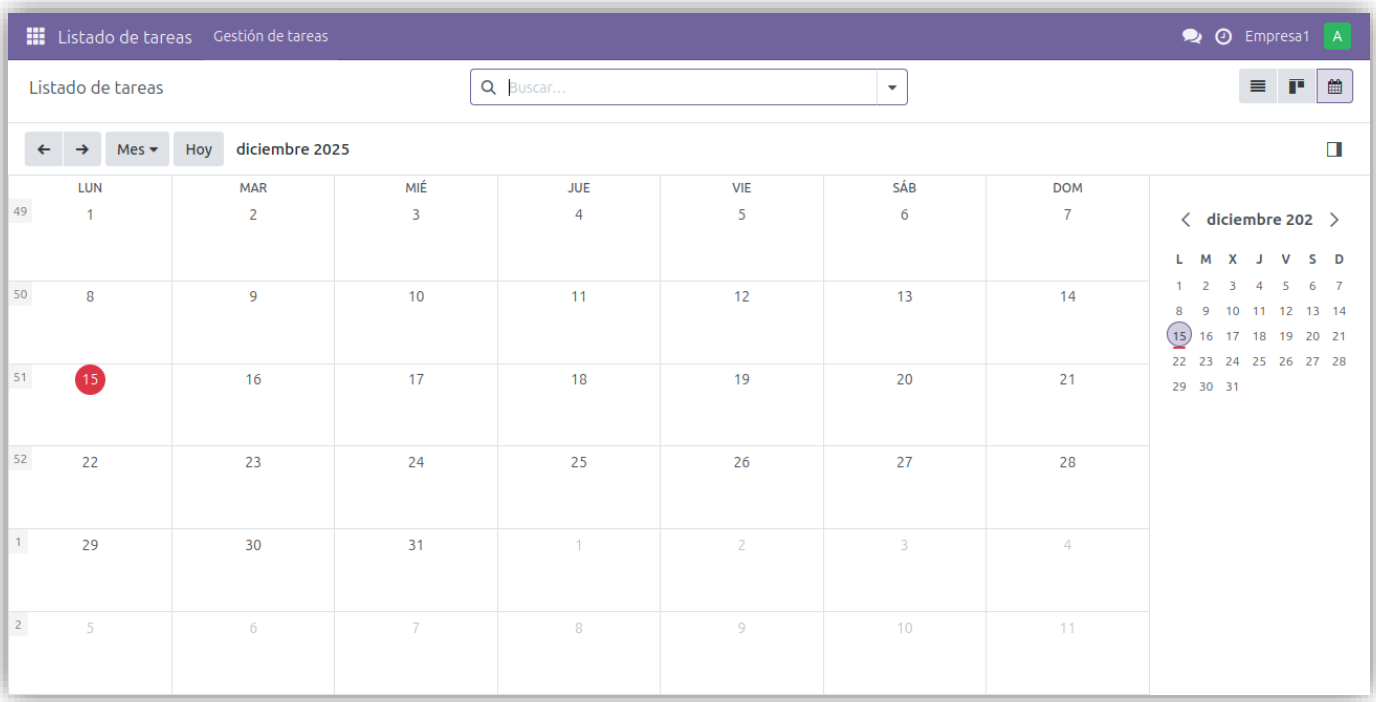
        <group string="Fechas">
          <field name="date_assigned"/>
        </group>
      </sheet>
    </form>
  </field>
</record>
```

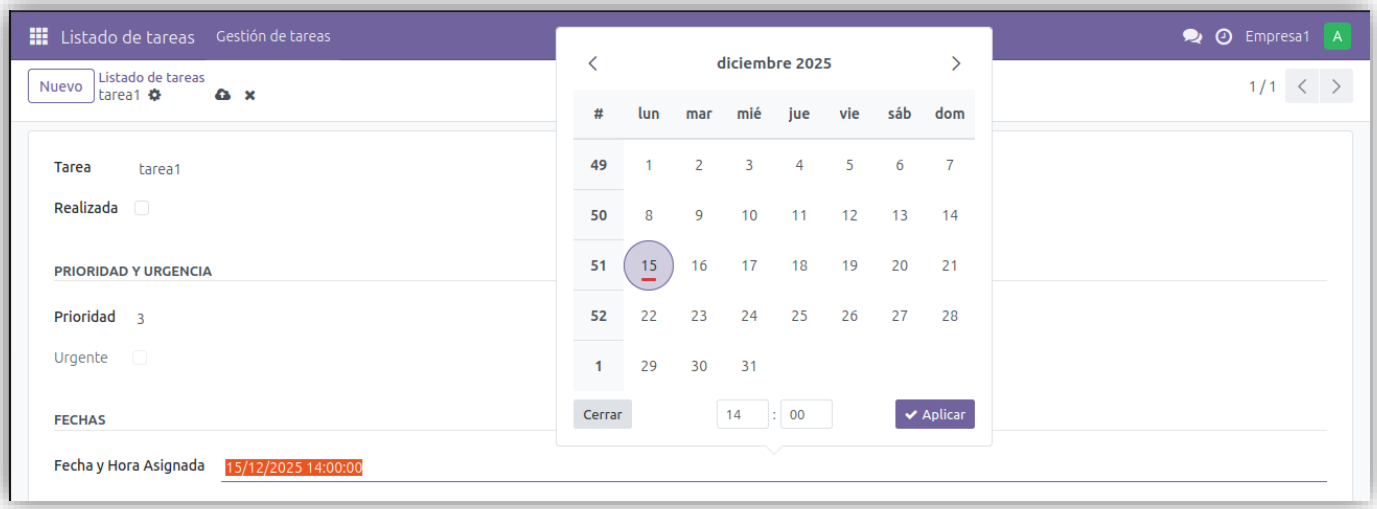
3. Crear/Confirmar la vista de Calendario XML para visualizar las tareas por fecha.
   # views.xml

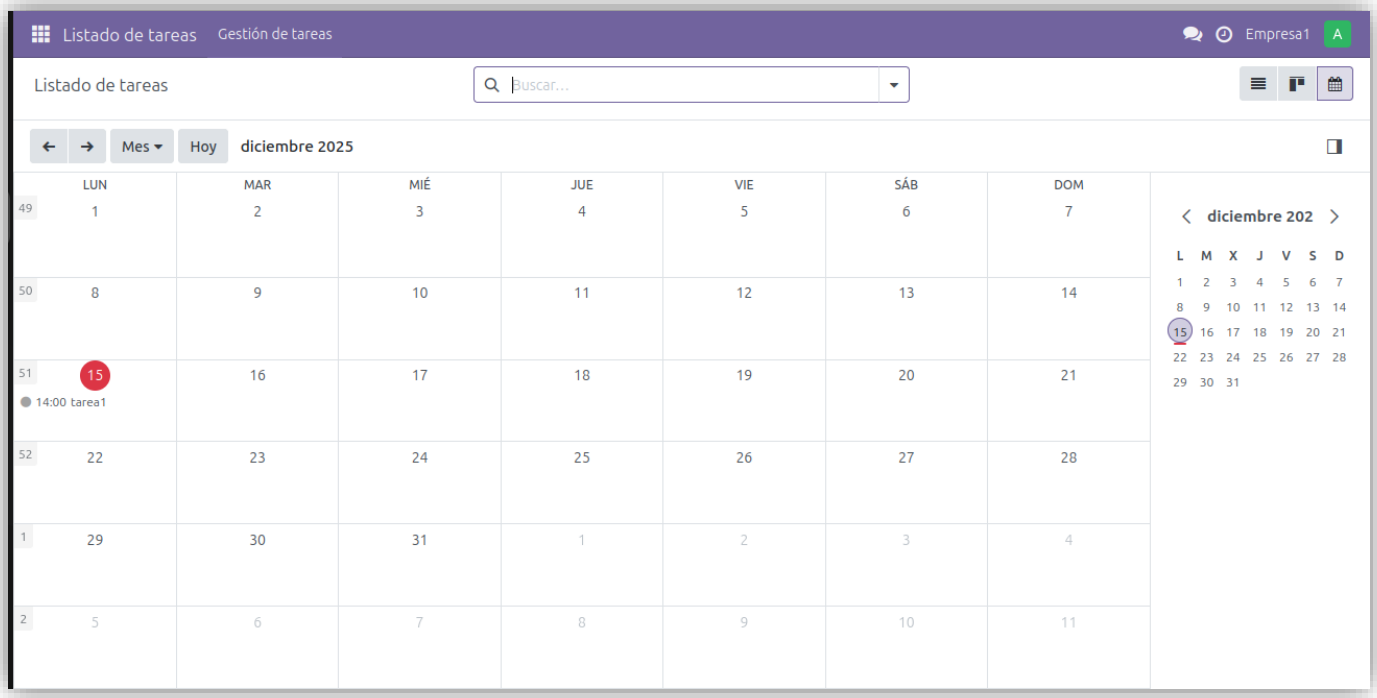
```
<record model="ir.ui.view" id="vista_calendario_lista_tarea">
    <field name="name">lista.tarea.calendar</field>
    <field name="model">lista_tareas.lista</field>
    <field name="arch" type="xml">
        <calendar string="Tareas Asignadas" date_start="date_assigned"
mode="month">
            <field name="tarea"/>
            <field name="prioridad"/>
        </calendar>
    </field>
</record>
```

- ✓ Guardar todos los archivos (.py y .xml).
- ✓ Reiniciar contenedor/servidor Odoo (p. ej., docker restart [ID de tu contenedor Odoo]).
- ✓ Actualiza el Módulo en la interfaz de Odoo (Menú Aplicaciones → Buscar módulo → Actualizar).






Listado de tareas
Gestión de tareas
Empresa1
Nuevo
Listado de tareas
tarea1
1 / 1
Tarea
tarea1
Realizada
PRIORIDAD Y URGENCIA
Prioridad
3
Urgente
FECHAS
Fecha y Hora Asignada
15/12/2025 14:00:00
diciembre 2025
lun mar mié jue vie sáb dom
49 1 2 3 4 5 6 7
50 8 9 10 11 12 13 14
51 15 16 17 18 19 20 21
52 22 23 24 25 26 27 28
1 29 30 31
Cerrar
14 : 00
Aplicar


Listado de tareas
Gestión de tareas
Empresa1
Listado de tareas
Buscar...
Mes
Hoy
diciembre 2025
LUN MAR MIÉ JUE VIE SÁB DOM
49 1 2 3 4 5 6 7
50 8 9 10 11 12 13 14
51 15 16 17 18 19 20 21
14:00 tarea1
52 22 23 24 25 26 27 28
1 29 30 31 1 2 3 4
2 5 6 7 8 9 10 11
diciembre 202
L M X J V S D
1 2 3 4 5 6 7
8 9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31

ACTIVIDAD 02

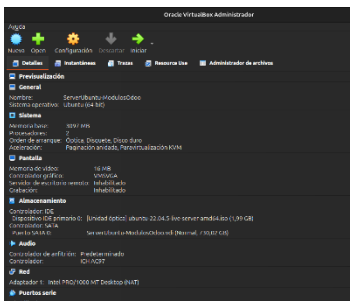
Amplía el módulo del ejemplo de biblioteca de cómics de forma que:

- Incluir la posibilidad de gestionar socios, almacenar nombre, apellido e identificador.
- Introduzca la posibilidad de que haya ejemplares de cómics a prestar.
- El modelo de cómic actual debe servir como referencia de la información del cómic. Aparte de este modelo, tendrá que plantear un nuevo modelo para indicar ejemplares de préstamo de ese cómic.
- Estos ejemplares de préstamo deberán controlar sólo a quienes están prestados y la fecha de inicio y fecha de final del préstamo. No es necesario tenerner un histórico de préstamos, sólo quien tiene el cómic en cada momento, cuando se lo ha prestado y la fecha prevista de regreso:
  - La fecha de préstamo no puede ser posterior al día de hoy.
  - La fecha prevista de regreso no puede ser anterior al día de hoy.

1. **Preparación del Servidor (VirtualBox)**
   - Configuración de Red: En VirtualBox, red en modo NAT. Reenvío de Puertos: Mapeamos los puertos :

Nombre	Protocolo	IP anfitrión	Puerto anfitrión	IP invitado	Puerto invitado
SSH	TCP	127.0.0.1	**2222**	(vacío)	**22**
Odoo	TCP	127.0.0.1	**8069**	(vacío)	**8069**

   - Acceso Remoto desde terminal usando: ssh -p 2222 miriam@127.0.0.1
   - Instalación de Docker
   - Crear el archivo docker-compose.yml con:
     - Imagen de Odoo 18.
     - Imagen de PostgreSQL 15.
     - Volúmenes: se crean la carpeta volumesOdoo/addons para que tus módulos no se borren.
   - Encendido: se levanta con docker compose up -d.

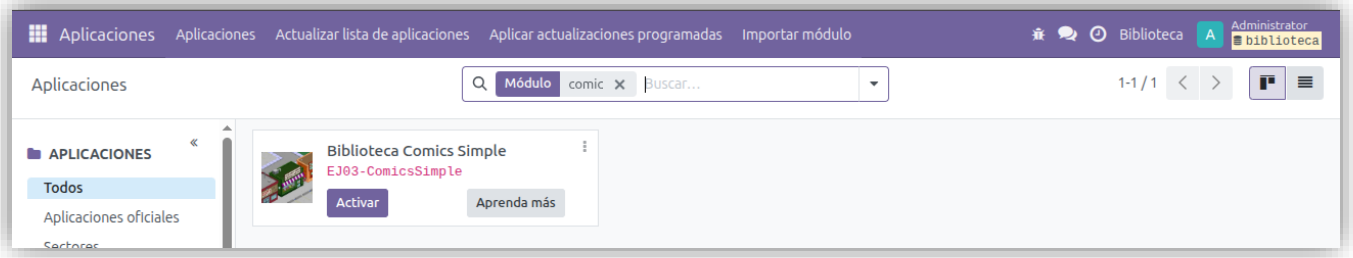


2. Implementación del Módulo EJ03-ComicsSimple

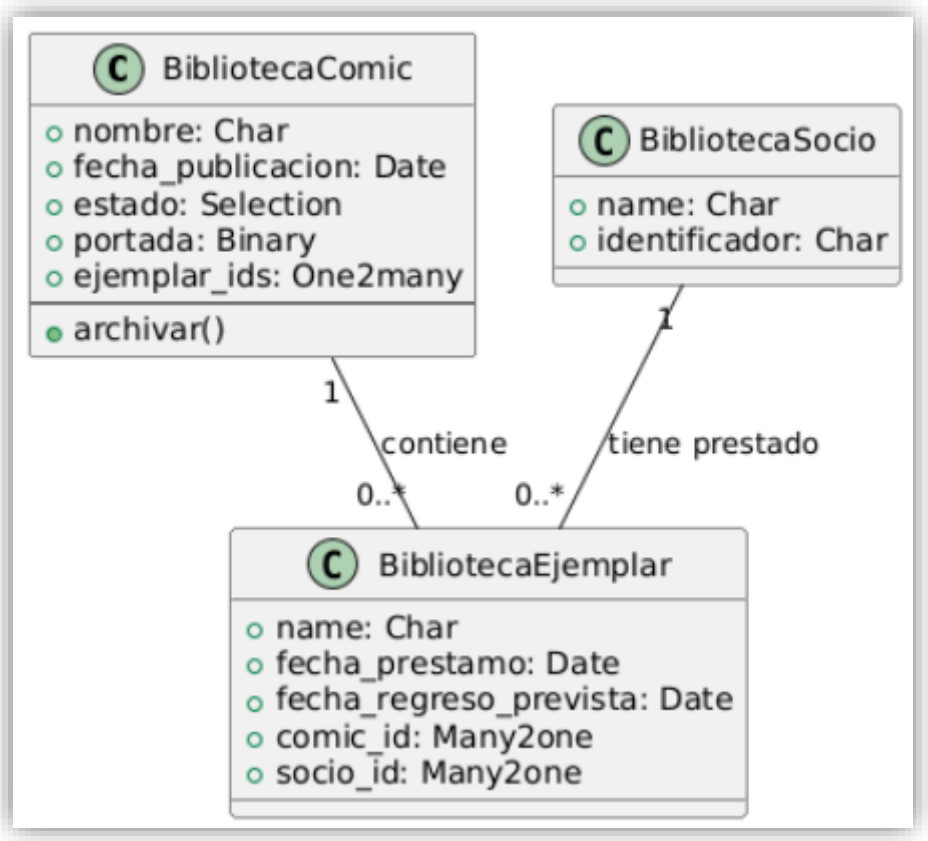
- Transferencia del Archivo (desde Anfitriona a VM) :
  scp -r -P 2222 "/ruta/absoluta/a/ EJ03-ComicsSimple " miriam@127.0.0.1:~/
- Mover el módulo: para llevar la carpeta del módulo desde tu home a la carpeta de addons de Odoo:
  sudo mv ~/EJ03-ComicsSimple ~/biblioteca/desarrollo/volumesOdoo/addons/
- Aplicar permisos: sudo chmod -R 777 para que Odoo no tuviera problemas al leer los archivos.



- Depuración de Código :
  - Error de Init: se borró from . import controllers porque la carpeta no existía.
  - Error de XML: se cambió las etiquetas <tree> por <list> porque Odoo 18 ya no se usa "tree" en las vistas de listado.



3. **Creación de dos nuevos modelos en Odoo:**
   - biblioteca.socio: Para almacenar la información de los socios (nombre, apellido e identificador).
   - biblioteca.ejemplar: Para representar los ejemplares físicos de un cómic que se pueden prestar. Este modelo tendrá una relación con el modelo de cómic (biblioteca.comic) para saber a qué cómic referencia, y campos para controlar el préstamo (socio prestado, fecha de inicio y fecha de fin prevista).
   - Se modifica el modelo existente biblioteca.comic para establecer una relación uno a muchos con los ejemplares.



Pasos de Desarrollo

- Definir el modelo **biblioteca.socio** (Python).
- Definir el modelo **biblioteca.ejemplar** con validaciones (Python).
- Modificar el modelo **biblioteca.comic** para incluir la relación con los ejemplares (Python).
- Crear las vistas para **biblioteca.socio** y **biblioteca.ejemplar** (XML/Vistas).
- Actualizar el menú para incluir la gestión de socios y ejemplares (XML/Menús).
- Actualizar las vistas de **biblioteca.comic** para mostrar la lista de ejemplares (XML/Vistas).

1) Definir el modelo biblioteca.socio

Este modelo almacena la identidad de las personas que pueden retirar cómics. Incluye un campo computado para facilitar la visualización del nombre completo.

#biblioteca_socio.py

```
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class BibliotecaSocio(models.Model):
    _name = 'biblioteca.socio'
    _description = 'Socio de la Biblioteca'
    _rec_name = 'nombre_completo'

    identificador = fields.Char('ID Socio', required=True)
    nombre = fields.Char('Nombre', required=True)
    apellido = fields.Char('Apellido', required=True)

    # Campo computado que une nombre y apellido
    nombre_completo = fields.Char(
        string='Socio',
        compute='_compute_nombre_completo',
        store=True
    )

    @api.depends('nombre', 'apellido')
    def _compute_nombre_completo(self):
        for record in self:
            record.nombre_completo = f"{record.nombre or ''} {record.apellido or
''}".strip()
```

2) Definir el modelo biblioteca.ejemplar con validaciones

Gestiona el estado del préstamo y contiene las reglas de negocio para las fechas.

biblioteca_ejemplar.py

```
# -*- coding: utf-8 -*-
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class BibliotecaEjemplar(models.Model):
    _name = 'biblioteca.ejemplar'
    _description = 'Ejemplar de Cómic'

    # Campo para identificar el ejemplar físico (ej: "Ejemplar 1")
    name = fields.Char(string='Nº Inventario', required=True)

    # Relación con el cómic base
    comic_id = fields.Many2one('biblioteca.comic', string='Cómic', required=True)

    # Control de préstamos
    socio_id = fields.Many2one('biblioteca.socio', string='Prestado a')
    fecha_prestamo = fields.Date(string='Fecha de Inicio')
    fecha_regreso_prevista = fields.Date(string='Fecha Prevista de Regreso')

    @api.constrains('fecha_prestamo')
    def _check_fecha_prestamo(self):
        for record in self:
            # La fecha de préstamo no puede ser futura
            if record.fecha_prestamo and record.fecha_prestamo > fields.Date.today():
                raise ValidationError("La fecha de préstamo no puede ser posterior a la fecha de hoy.")

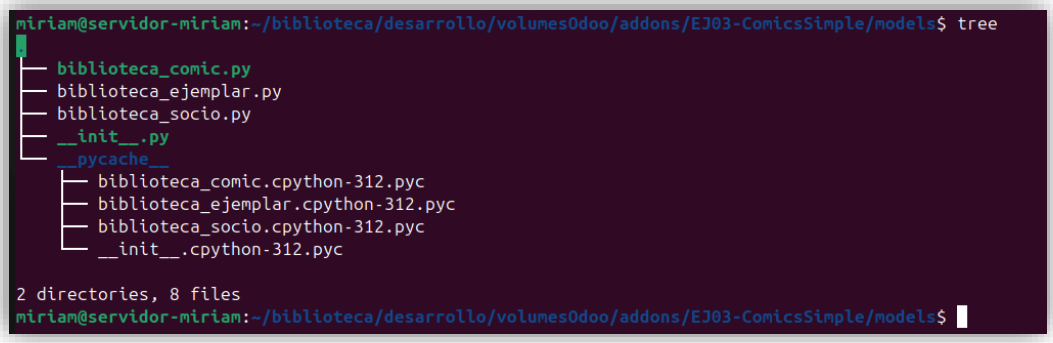
    @api.constrains('fecha_regreso_prevista')
    def _check_fecha_regreso(self):
        for record in self:
            # La fecha de regreso no puede ser pasada
            if record.fecha_regreso_prevista and record.fecha_regreso_prevista < fields.Date.today():
                raise ValidationError("La fecha prevista de regreso no puede ser anterior a la fecha de hoy.")
```

3) Modificar biblioteca.comic (se agrega código)

Para que desde la ficha de un Cómic cuántos ejemplares físicos hay, se añade una relación inversa One2many.

#/models/biblioteca_comic.py

```
# Permite acceder a los ejemplares desde el cómic
    ejemplar_ids = fields.One2many(
        'biblioteca.ejemplar',
        'comic_id',
        string='Ejemplares'
    )
```



4) Vistas y Menús (XML)

En el archivo views/views.xml, se define cómo se ven estos datos y cómo navegar entre ellos.

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>

  <record id='biblioteca_comic_action' model='ir.actions.act_window'>
    <field name="name">Biblioteca de Comics</field>
    <field name="res_model">biblioteca.comic</field>
    <field name="view_mode">list,form,kanban,calendar</field>
  </record>

  <record id="action_biblioteca_socio" model="ir.actions.act_window">
    <field name="name">Socios</field>
    <field name="res_model">biblioteca.socio</field>
    <field name="view_mode">list,form,kanban</field>
  </record>

  <record id="action_biblioteca_ejemplar" model="ir.actions.act_window">
    <field name="name">Ejemplares</field>
    <field name="res_model">biblioteca.ejemplar</field>
    <field name="view_mode">list,calendar,form</field>
  </record>

  <record id="biblioteca_comic_view_form" model="ir.ui.view">
    <field name="name">biblioteca.comic.form</field>
    <field name="model">biblioteca.comic</field>
    <field name="arch" type="xml">
      <form>
```

```
        <header>
          <button type="object" name="archivar" string="Archivar Comics" class="btn-primary"/>
        </header>
        <sheet>
          <group>
            <group>
              <field name="nombre"/>
              <field name="autor_ids" widget="many2many_tags"/>
              <field name="estado"/>
              <field name="paginas"/>
              <field name="activo" readonly="1"/>
            </group>
            <group>
              <field name="precio"/>
              <field name="fecha_publicacion"/>
              <field name="portada" widget="image" class="oe_avatar"/>
              <field name="valoracion_lector"/>
            </group>
          </group>
          <notebook>
            <page string="Descripción">
              <group>
                <field name="descripcion"/>
              </group>
            </page>
            <page string="Ejemplares">
              <field name="ejemplar_ids">
                <list editable="bottom">
                  <field name="name"/>
                  <field name="socio_id"/>
                  <field name="fecha_prestamo"/>
                  <field name="fecha_regreso_prevista"/>
                </list>
              </field>
            </page>
          </notebook>
        </sheet>
      </form>
    </field>
  </record>
```

```
<record id="biblioteca_comic_view_tree" model="ir.ui.view">
  <field name="name">biblioteca.comic.list</field>
  <field name="model">biblioteca.comic</field>
  <field name="arch" type="xml">
    <list>
      <field name="nombre"/>
      <field name="fecha_publicacion"/>
      <field name="estado"/>
    </list>
  </field>
</record>

<record id="biblioteca_comic_view_kanban" model="ir.ui.view">
  <field name="name">biblioteca.comic.kanban</field>
  <field name="model">biblioteca.comic</field>
  <field name="arch" type="xml">
    <kanban>
      <field name="id"/>
      <templates>
        <t t-name="kanban-box">
          <div class="oe_kanban_global_click">
            <div class="o_kanban_image">
              <img t-att-src="kanban_image('biblioteca.comic', 'portada', record.id.raw_value)"
alt="Portada"/>
            </div>
            <div class="oe_kanban_details">
              <strong><field name="nombre"/></strong>
              <div><field name="autor_ids"/></div>
            </div>
          </div>
        </t>
      </templates>
    </kanban>
  </field>
</record>
```

```
<record id="biblioteca_socio_view_form" model="ir.ui.view">
    <field name="name">biblioteca.socio.form</field>
    <field name="model">biblioteca.socio</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <group>
                    <group>
                        <field name="nombre"/>
                        <field name="apellido"/>
                        <field name="identificador"/>
                    </group>
                    <group>
                        <field name="nombre_completo" readonly="1"/>
                    </group>
                </group>
            </sheet>
        </form>
    </field>
</record>

<record id="biblioteca_socio_view_tree" model="ir.ui.view">
    <field name="name">biblioteca.socio.list</field>
    <field name="model">biblioteca.socio</field>
    <field name="arch" type="xml">
        <list>
            <field name="identificador"/>
            <field name="nombre_completo"/>
        </list>
    </field>
</record>
```

```
  <record id="biblioteca_socio_view_kanban" model="ir.ui.view">
    <field name="name">biblioteca.socio.kanban</field>
    <field name="model">biblioteca.socio</field>
    <field name="arch" type="xml">
      <kanban>
        <templates>
          <t t-name="kanban-box">
            <div class="oe_kanban_global_click">
              <div class="oe_kanban_details">
                <strong><field name="nombre_completo"/></strong>
                <div>ID: <field name="identificador"/></div>
              </div>
            </div>
          </t>
        </templates>
      </kanban>
    </field>
  </record>

  <record id="biblioteca_ejemplar_view_calendar" model="ir.ui.view">
    <field name="name">biblioteca.ejemplar.calendar</field>
    <field name="model">biblioteca.ejemplar</field>
    <field name="arch" type="xml">
      <calendar string="Préstamos"
              date_start="fecha_prestamo"
              date_stop="fecha_regreso_prevista"
              color="socio_id">
        <field name="name"/>
        <field name="socio_id"/>
      </calendar>
    </field>
  </record>
  <menuitem name="Mi biblioteca" id="biblioteca_base_menu" />
    <menuitem name="Comics" id="biblioteca_comic_menu"
        parent="biblioteca_base_menu"
        action="biblioteca_comic_action" sequence="10"/>
  <menuitem name="Socios" id="menu_socio"
        parent="biblioteca_base_menu"
        action="action_biblioteca_socio" sequence="20"/>
   <menuitem name="Ejemplares" id="menu_ejemplar"
        parent="biblioteca_base_menu"
        action="action_biblioteca_ejemplar" sequence="30"/>
</odoo>
```

5) Grupos de Seguridad (groups.xml)

Antes de dar permisos en el CSV, crear Grupos. Esto permite diferenciar, por ejemplo, entre un "Bibliotecario" (que puede borrar cómics) y un "Lector" (que solo puede verlos).

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <!-- Aqui definimos como se forma el grupo de seguridad "grupo_bibliotecario"
    basado en "res.groups", que es el modelo de los grupos de Odoo -->
  <record id="grupo_bibliotecario" model="res.groups">
    <!-- El field "name" indica como se mostrará en la interfaz.
      En este caso se verá como "Bibliotecario" -->
    <field name="name">Biblotecario</field>
    <!-- El field "users" indica que los usuarios de este grupo se incluyan en este
      grupo de seguirdad.
      En este caso con "base.user_admin" estamos incluyendo a los administradores -->
    <field name="users" eval="[(4, ref('base.user_admin'))]" />
    <!-- Para otros casos, podriamos incluir a un grupo creado en Odoo que nos interese
      (jefes, encargados, empleados, etc.) -->
  </record>
</odoo>
```

Se ha creado un grupo de seguridad específico llamado Bibliotecario.
Modelo de referencia: res.groups.
Asignación automática: Se ha configurado para que el usuario administrador de Odoo (base.user_admin) pertenezca a este grupo por defecto mediante el comando (4, ref('base.user_admin')).
Este grupo actúa como la "llave" para otorgar permisos de edición en todo el módulo.

6) Permisos (ir.model.access.csv)

Para regular qué puede hacer cada grupo en cada tabla (modelo), se ha definido una lista de control de acceso (ACL). Los permisos se dividen en cuatro categorías: Read (Leer), Write (Escribir), Create (Crear) y Unlink (Borrar).

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_biblioteca_comic_admin,biblioteca.comic.admin,model_biblioteca_comic,EJ03-ComicsSim-
ple.grupo_bibliotecario,1,1,1,1
access_biblioteca_socio_admin,biblioteca.socio.admin,model_biblioteca_socio,EJ03-
ComicsSimple.grupo_bibliotecario,1,1,1,1
access_biblioteca_ejemplar_admin,biblioteca.ejemplar.admin,model_biblioteca_ejemplar,EJ03-
ComicsSimple.grupo_bibliotecario,1,1,>
access_biblioteca_comic_public,biblioteca.comic.public,model_biblioteca_comic,base.group_user,1,0,0,0
```

Este archivo define los derechos de acceso utilizando una estructura de columnas específica:

- id: Identificador único de la regla.
- name: Nombre descriptivo (usado para depuración).
- model_id:id: El modelo técnico sobre el que se aplica la regla (prefijo model_).
- group_id:id: El grupo de usuarios que recibirá estos permisos.
- perm_read, perm_write, perm_create, perm_unlink: Permisos de Lectura, Escritura, Creación y Borrado (1 = Permitido, 0 = Denegado).

➢ Administración de Cómics

access_biblioteca_comic_admin,biblioteca.comic.admin,model_biblioteca_comic,EJ03-ComicsSimple.grupo_bibliotecario,1,1,1,1

Otorga al grupo Bibliotecario control total sobre el catálogo de cómics. Pueden verlos, editarlos, crear nuevos y borrarlos.

➢ Administración de Socios

access_biblioteca_socio_admin,biblioteca.socio.admin,model_biblioteca_socio,EJ03-ComicsSimple.grupo_bibliotecario,1,1,1,1

El Bibliotecario tiene permisos totales sobre el modelo de socios. Esto es necesario para dar de alta a nuevos lectores y gestionar sus datos personales.

- Administración de Ejemplares

  access_biblioteca_ejemplar_admin,biblioteca.ejemplar.admin,model_biblioteca_ejemplar,EJ03-ComicsSimple.grupo_bibliotecario,1,1,1,0

  El Bibliotecario puede crear y editar ejemplares (para gestionar préstamos), pero no tiene permiso para borrarlos. Esto protege el historial de la biblioteca; si un ejemplar existe, debe quedar registro de él.

- Acceso Público (Lectura)

  access_biblioteca_comic_public,biblioteca.comic.public,model_biblioteca_comic,base.group_user,1,0,0,0

  para el resto de los usuarios de Odoo (base.group_user). Solo tienen permiso de Lectura (1). Pueden entrar y ver qué cómics hay disponibles, pero no pueden cambiar precios, nombres ni borrar nada (0,0,0).

7) Comandos Esenciales utilizados para Odoo y Docker

❖ Actualización del Módulo

docker exec -u 0 desarrollo-web-1 odoo -u EJ03-ComicsSimple -d odoo --db_host db --db_user odoo --db_password odoo --stop-after-init

Este comando obliga a Odoo a leer de nuevo tus archivos Python, XML y CSV. Se usa cada vez que haces un cambio en el código.

Parámetro -u: Indica el nombre técnico de tu módulo (EJ03-ComicsSimple).

Parámetro --stop-after-init: Hace que el contenedor procese el cambio y se detenga, asegurando que los cambios se guarden en la base de datos de forma limpia.

❖ Reinicio del Contenedor

docker restart desarrollo-web-1

Reinicia el servicio de Odoo por completo. Es útil cuando el sistema se queda "con gelado" o cuando hay cambios profundos en la configuración del servidor que no se aplican solo con actualizar el módulo.

❖ Visualización de Logs en Tiempo Real

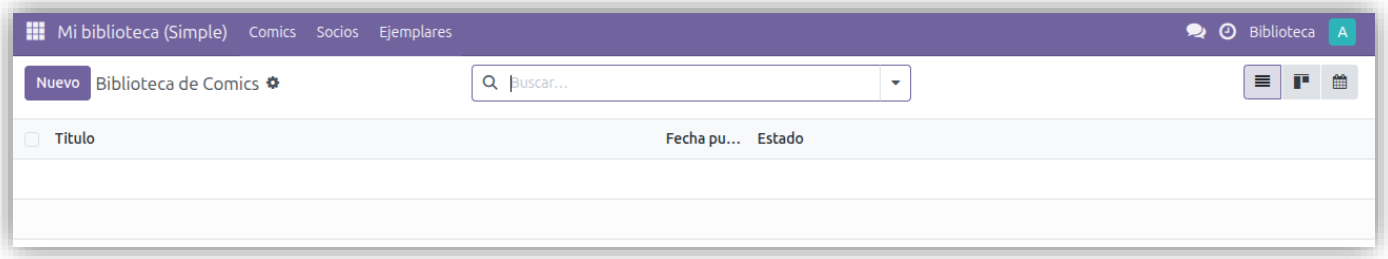
docker logs -f desarrollo-web-1

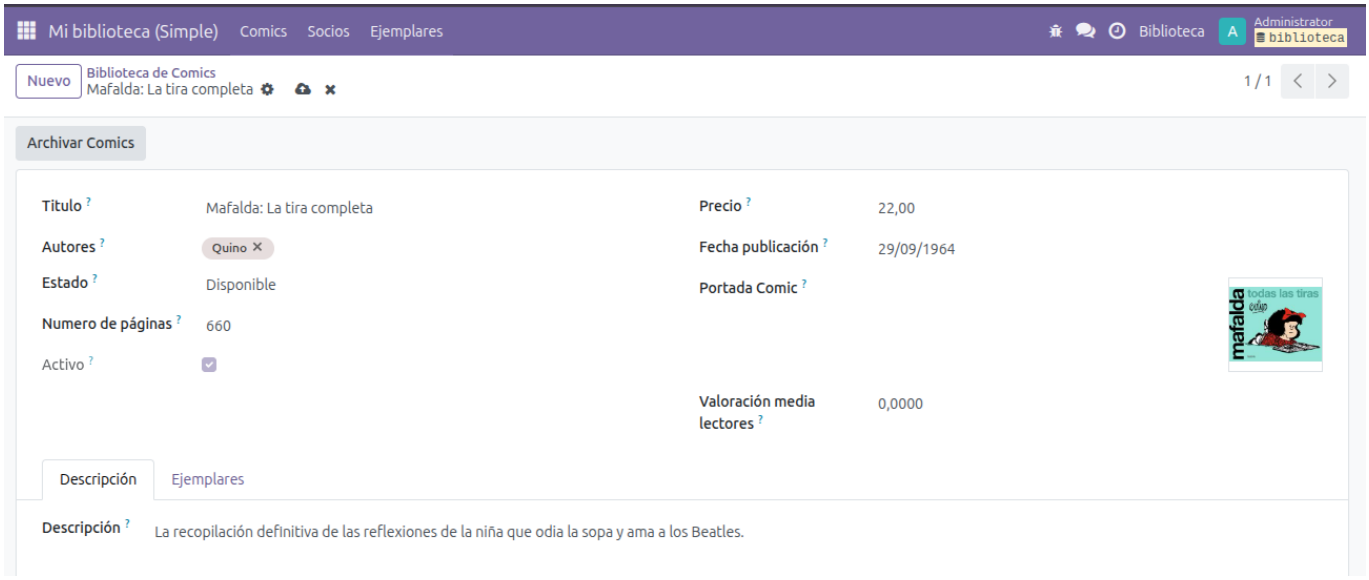
Permite ver qué está pasando , vital para el debugging (depuración): si hay un error en el código Python, el error aparecerá aquí con una descripción detallada (Trace back).

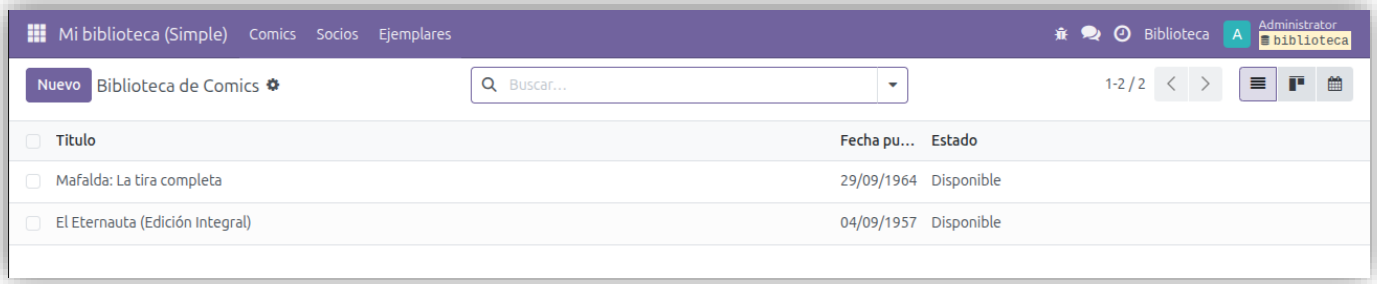
❖ Instalación de un Módulo Nuevo

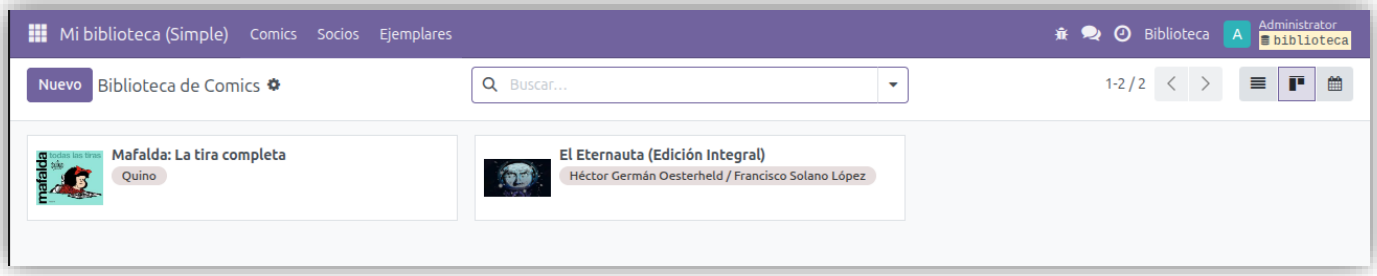
docker exec -u 0 desarrollo-web-1 odoo -i EJ03-ComicsSimple -d odoo --db_host db --db_user odoo --db_password odoo --stop-after-init

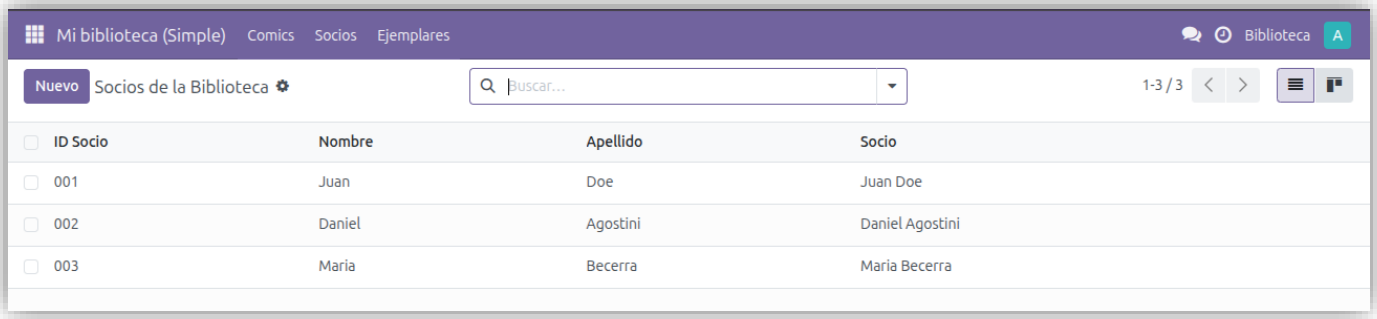
Similar al comando de actualización, pero usa el parámetro -i (Install). Se usa la pri mera vez que subes el módulo al servidor para que Odoo cree las tablas en la base de datos.

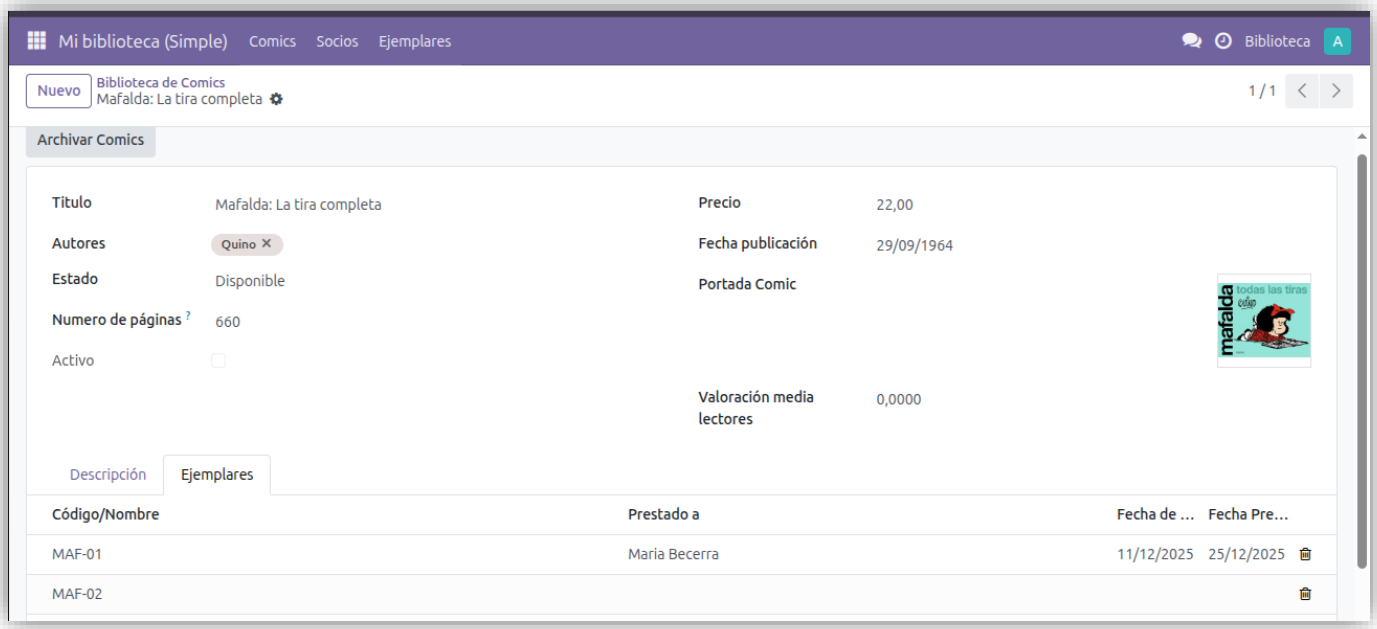





Mi biblioteca (Simple)
Comics
Socios
Ejemplares
Biblioteca
Administrator
biblioteca
Nuevo
Biblioteca de Comics
Buscar...
1-2 / 2
Titulo
Fecha pu...
Estado
Mafalda: La tira completa
29/09/1964
Disponible
El Eternauta (Edición Integral)
04/09/1957
Disponible


Mi biblioteca (Simple)
Comics
Socios
Ejemplares
Biblioteca
Administrator
biblioteca
Nuevo
Biblioteca de Comics
Buscar...
1-2 / 2
Mafalda: La tira completa
Quino
El Eternauta (Edición Integral)
Héctor Germán Oesterheld / Francisco Solano López


Mi biblioteca (Simple)
Comics
Socios
Ejemplares
Biblioteca
Nuevo
Socios de la Biblioteca
Buscar...
1-3 / 3
ID Socio
Nombre
Apellido
Socio
001
Juan
Doe
Juan Doe
002
Daniel
Agostini
Daniel Agostini
003
Maria
Becerra
Maria Becerra


Mi biblioteca (Simple)
Comics
Socios
Ejemplares
Biblioteca
Nuevo
Biblioteca de Comics
Mafalda: La tira completa
1 / 1
Archivar Comics
Titulo
Mafalda: La tira completa
Autores
Quino
Estado
Disponible
Numero de páginas
660
Activo
Precio
22,00
Fecha publicación
29/09/1964
Portada Comic
Valoración media lectores
0,0000
Descripción
Ejemplares
Código/Nombre
Prestado a
Fecha de ...
Fecha Pre...
MAF-01
Maria Becerra
11/12/2025
25/12/2025
MAF-02

ACTIVIDAD 03

Crea un módulo para gestionar a pacientes y médicos de un hospital.

- Por cada paciente, tendremos un modelo con los siguientes datos:
  - Nombre y apellidos del paciente.
  - Síntomas

- Por cada médico, tendremos un modelo con los siguientes datos:
  - Nombre y apellidos del médico.
  - Número de colegiado.

- Por cada vez que un médico ha atendido a un paciente, tendremos un modelo indicando el diagnóstico.

- Un paciente puede haber sido atendido por varios médicos y un médico puede haber atendido a varios pacientes.

Implementa los modelos y vistas que creas adecuadas para los 3 modelos.

---

1) **Crear Estructura del proyecto:**

   sudo chmod -R 777 . (en addons)

2) ***Generar la estructura (Scaffold)***
   Se ejecuta el comando desde la terminal en donde tenemos el compose para la creación de carpeta y archivos base:

   docker compose exec -u root web odoo scaffold hospital_gestion /mnt/extra-addons

3) ***Verificación de archivos: tree***
   Estructura generada
   - __manifest__.py: El archivo de identidad del módulo (nombre, versión, dependencias).
   - models/: Donde definirás las tablas de la base de datos (Python).
   - views/: Donde definirás la interfaz (XML).
   - security/: El archivo ir.model.access.csv para dar permisos a los modelos.
   - controllers/ y demo/: Para rutas web y datos de prueba.

4) ***Ajuste de permisos***

Para poder editar los archivos sin problemas de "Acceso denegado":

sudo chown -R $USER:$USER ./volumesOdoo/addons/hospital_gestion
Docker compose restart web (reinicio contenedor para se detecte el módulo)

5) ***Borrado de carpeta generada por defecto no utilizadas:***

rm -rf controller/ demo/

borrar referencias de dichas carpetas en el init y manifest

6) **Configurar el manifiesto**

Nano __manifest__.py para definir el módulo:

```
# -*- coding: utf-8 -*-
{
    'name': "hospital_gestion",

    'summary': "Módulo para gestionar pacientes, médicos y consultas.",

    'description': """
        Este módulo permite:
        - Registrar pacientes.
        - Gestionar médicos y sus números de colegiado.
        - Registrar consulta con su fecha.
    """,

    'author': "Miriam Mendieta",
    'website': "https://www.tu-sitio.com",

    # Categoría para la clasificación en el panel de Odoo
    'category': 'Uncategorized',
    'version': '0.1',

    # Cualquier módulo necesario para que este funcione (base es el estándar)
    'depends': ['base'],

    # se carga primero la seguridad y luego las vistas
    'data': [
        'security/ir.model.access.csv',
        'views/views.xml',
    ],

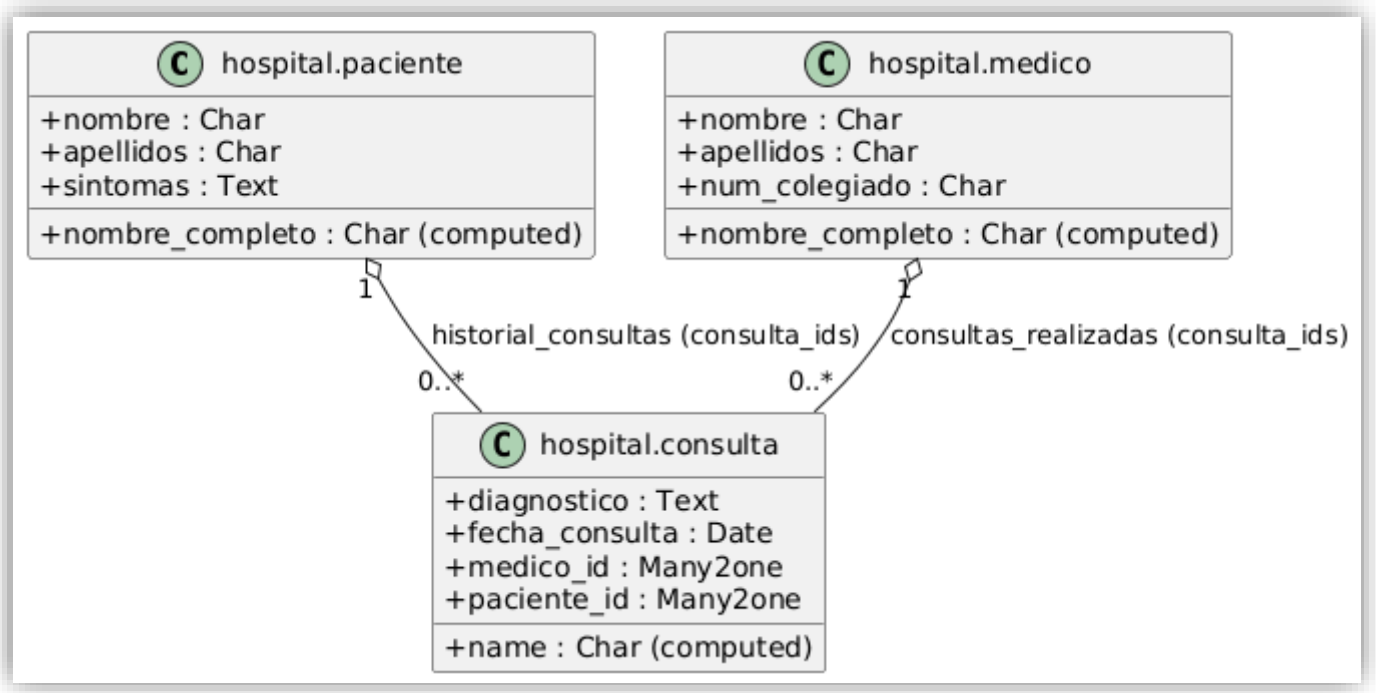
    'installable': True,
    'application': True,
}
```

7) **Instalar el módulo**

- ✓ Reinicia contenedor: docker restart nombre_contenedor.
- ✓ Entra a Odoo en el navegador con el Modo Desarrollador activado →Aplicaciones →clic en Actualizar lista de aplicaciones.
- ✓ Buscar modulo e instalarlo.

8) **Creación de tres modelos en Python:**

- hospital.paciente: Almacena los datos básicos del paciente.
- hospital.medico: Almacena los datos del personal médico.
- hospital.consulta: El modelo relacional que vincula a un médico con un paciente y guarda el diagnóstico.



9) **Pasos de desarrollo:**

- ✓ Definir las clases en models.py.
- ✓ Configurar los permisos de acceso en ir.model.access.csv.
- ✓ Crear las vistas (formularios y listas) en XML.

```
miriam@servidor-miriam:~/hospital/desarrollo$ cd volumesOdoo/addons/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons$ ls
hospital_gestion
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons$ cd hospital_gestion/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ ls
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ ls
controllers  demo  __init__.py  __manifest__.py  models  security  views
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ rm -rf controllers/ demo/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ ls
__init__.py  __manifest__.py  models  security  views
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ nano __init__.py
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ nano __manifest__.py
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ cd views/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$ ls
templates.xml  views.xml
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$ rm templates.xml
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$ ls
views.xml
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$
```

10) Definición de los Modelos (models/models.py)

```
# -*- coding: utf-8 -*-
from odoo import models, fields, api

class HospitalPaciente(models.Model):
    _name = 'hospital.paciente'
    _description = 'Paciente'
    _rec_name = 'nombre_completo'

    nombre = fields.Char(string='Nombre', required=True)
    apellido = fields.Char(string='Apellido', required=True)
    nombre_completo = fields.Char(string='Nombre Completo', compute='_compute_nombre_completo', store=True)
    sintomas = fields.Text(string='Síntomas')

    @api.depends('nombre', 'apellido')
    def _compute_nombre_completo(self):
        for record in self:
            nom = record.nombre or ""
            ape = record.apellido or ""
            record.nombre_completo = f"{nom} {ape}".strip()
```

```
class HospitalMedico(models.Model):
    _name = 'hospital.medico'
    _description = 'Médico'
    _rec_name = 'nombre_completo'

    nombre = fields.Char(string='Nombre', required=True)
    apellido = fields.Char(string='Apellido', required=True)
    nombre_completo = fields.Char(string='Nombre Completo', compute='_compute_nombre_completo', store=True)
    numero_colegiado = fields.Char(string='Número de Colegiado', required=True)

    @api.depends('nombre', 'apellido')
    def _compute_nombre_completo(self):
        for record in self:
            nom = record.nombre or ""
            ape = record.apellido or ""
            record.nombre_completo = f"{nom} {ape}".strip()

class HospitalConsulta(models.Model):
    _name = 'hospital.consulta'
    _description = 'Consulta Médica'
    _rec_name = 'paciente_id' # Muestra el nombre del paciente como título de la consulta

    diagnostico = fields.Text(string='Diagnóstico')
    fecha_consulta = fields.Datetime(string='Fecha de Consulta', default=fields.Datetime.now)
    medico_id = fields.Many2one('hospital.medico', string='Médico', required=True)
    paciente_id = fields.Many2one('hospital.paciente', string='Paciente', required=True)
    state = fields.Selection([
        ('borrador', 'Borrador'),
        ('confirmada', 'Confirmada'),
        ('finalizada', 'Finalizada'),
    ], string='Estado', default='borrador')
```

Desde el punto de vista de la ingeniería de datos y la gestión hospitalaria, incluir el campo fecha_consulta en el modelo hospital.consulta es una necesidad técnica y estructural, ya que los datos médicos deben garantizar la inmutabilidad y la reconstrucción cronológica. La fecha permite establecer una secuencia lógica en el historial del paciente; sin este atributo, el sistema perdería la capacidad de discernir la evolución de patologías, invalidando la utilidad del historial clínico como herramienta de diagnóstico y soporte legal.

El diagrama de clases especifica una relación de uno a muchos (1..N) entre las entidades principales (Médico/Paciente) y la entidad relacional (Consulta). En la base de datos, la fecha actúa como un atributo discriminante. Sin ella, múltiples interacciones entre los mismos actores resultarían en registros semánticamente indistinguibles, dificultando la identificación única de cada evento asistencial en el tiempo.

La fecha permite:

- Analizar la carga asistencial y la optimización de recursos humanos.
- Monitorizar indicadores de salud pública.
- Gestionar ciclos de facturación y cumplimiento de normativas administrativas.

11) Actualizar la Seguridad (security/ir.model.access.csv)

Se deben registrar los 3 modelos:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_hospital_paciente,hospital.paciente,model_hospital_paciente,base.group_user,1,1,1,1
access_hospital_medico,hospital.medico,model_hospital_medico,base.group_user,1,1,1,1
access_hospital_consulta,hospital.consulta,model_hospital_consulta,base.group_user,1,1,1,1
```

12) Creación las vistas (formularios, listas, calendar, kanban) en XML.

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <data>
    <record id="view_hospital_paciente_search" model="ir.ui.view">
      <field name="name">hospital.paciente.search</field>
      <field name="model">hospital.paciente</field>
      <field name="arch" type="xml">
        <search>
          <field name="nombre_completo"/>
          <field name="sintomas"/>
        </search>
      </field>
    </record>

    <record id="view_hospital_paciente_list" model="ir.ui.view">
      <field name="name">hospital.paciente.list</field>
      <field name="model">hospital.paciente</field>
      <field name="arch" type="xml">
        <list>
          <field name="nombre_completo"/>
          <field name="sintomas"/>
        </list>
      </field>
    </record>
```

```
<record id="view_hospital_paciente_form" model="ir.ui.view">
    <field name="name">hospital.paciente.form</field>
    <field name="model">hospital.paciente</field>
    <field name="arch" type="xml">
      <form>
        <sheet>
          <group>
            <field name="nombre"/>
            <field name="apellido"/>
            <field name="sintomas"/>
          </group>
        </sheet>
      </form>
    </field>
  </record>

  <record id="view_hospital_medico_search" model="ir.ui.view">
    <field name="name">hospital.medico.search</field>
    <field name="model">hospital.medico</field>
    <field name="arch" type="xml">
      <search>
        <field name="nombre_completo"/>
        <field name="numero_colegiado"/>
      </search>
    </field>
  </record>
<record id="view_hospital_medico_list" model="ir.ui.view">
    <field name="name">hospital.medico.list</field>
    <field name="model">hospital.medico</field>
    <field name="arch" type="xml">
      <list>
        <field name="nombre_completo"/>
        <field name="numero_colegiado"/>
      </list>
    </field>
  </record>

<record id="view_hospital_medico_kanban" model="ir.ui.view">
    <field name="name">hospital.medico.kanban</field>
    <field name="model">hospital.medico</field>
    <field name="arch" type="xml">
      <kanban>
        <templates>
          <t t-name="kanban-box">
            <div class="oe_kanban_global_click">
              <strong><field name="nombre_completo"/></strong>
              <div>Colegiado: <field name="numero_colegiado"/></div>
            </div>
```

```
                </t>
            </templates>
        </kanban>
    </field>
</record>

<record id="view_hospital_consulta_form" model="ir.ui.view">
    <field name="name">hospital.consulta.form</field>
    <field name="model">hospital.consulta</field>
    <field name="arch" type="xml">
        <form>
            <header>
                <field name="state" widget="statusbar" options="{'clickable': '1'}"/>
            </header>
            <sheet>
                <group>
                    <group>
                        <field name="paciente_id"/>
                        <field name="medico_id"/>
                    </group>
                    <group>
                        <field name="fecha_consulta"/>
                    </group>
                </group>
                <notebook>
                    <page string="Diagnóstico">
                        <field name="diagnostico"/>
                    </page>
                </notebook>
            </sheet>
        </form>
    </field>
</record>

<record id="action_hospital_paciente" model="ir.actions.act_window">
    <field name="name">Pacientes</field>
    <field name="res_model">hospital.paciente</field>
    <field name="view_mode">list,form</field>
</record>

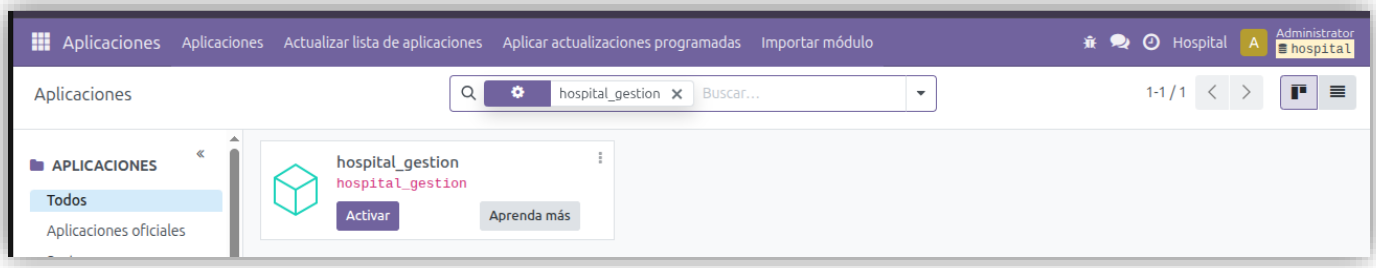
<record id="action_hospital_medico" model="ir.actions.act_window">
    <field name="name">Médicos</field>
    <field name="res_model">hospital.medico</field>
    <field name="view_mode">kanban,list,form</field>
</record>
```

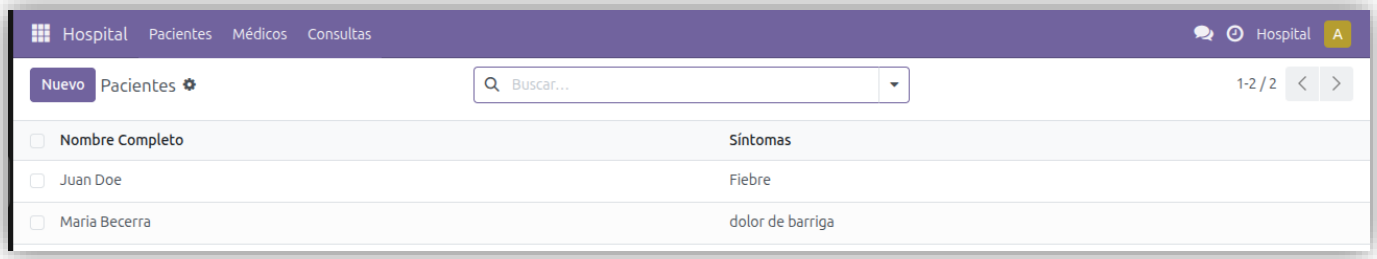
```
    <menuitem id="menu_hospital_root" name="Hospital"/>
    <menuitem id="menu_hospital_paciente" name="Pacientes" parent="menu_hospital_root" action="action_hospital_pa
        ciente" sequence="10"/>
    <menuitem id="menu_hospital_medico" name="Médicos" parent="menu_hospital_root" action="action_hospital_medico"
        sequence="20"/>
    <menuitem id="menu_hospital_consulta" name="Consultas" parent="menu_hospital_root" action="action_hospital_con
        sulta" sequence="30"/>
  </data>
</odoo>
```

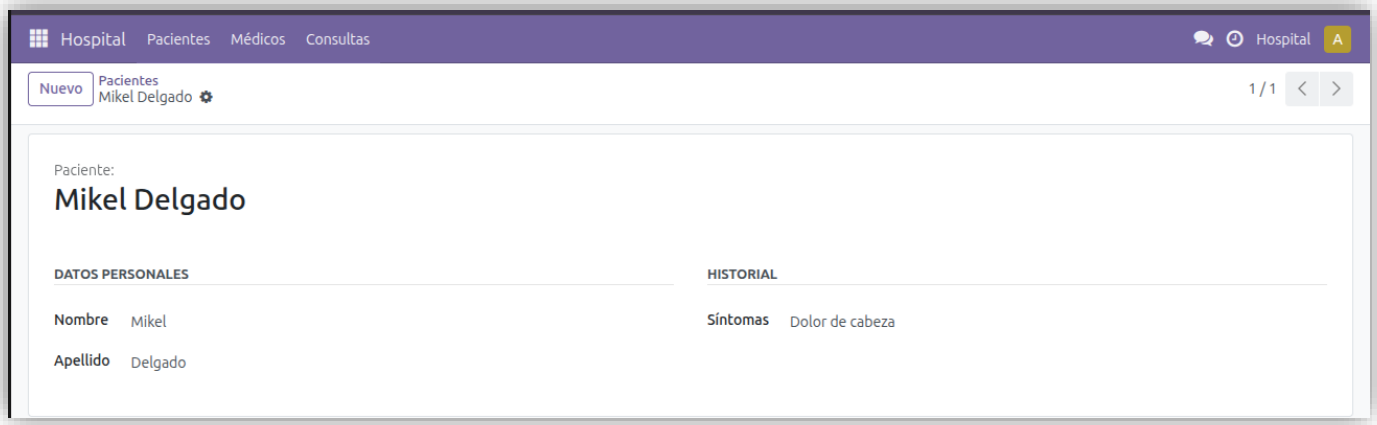
13)Actualizar Odoo:

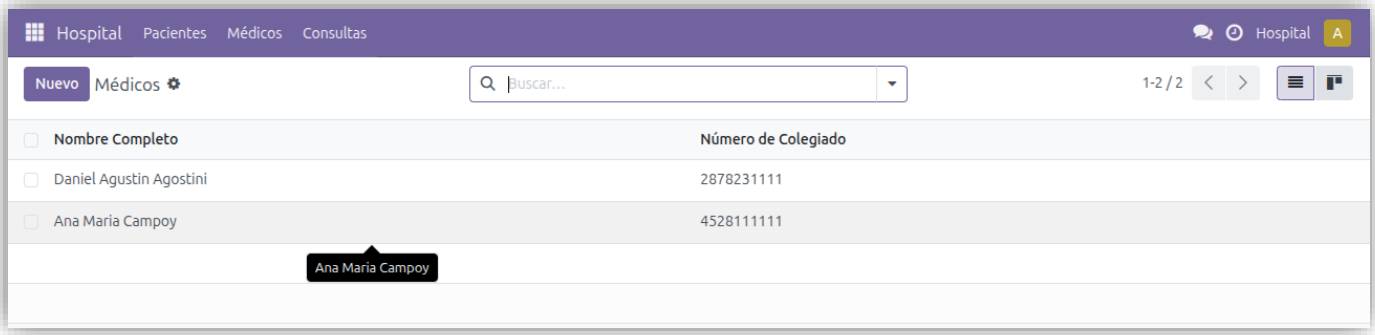
- Reinicia el contenedor: docker restart nombre_del_contenedor.
- Actualiza el módulo: ir interfaz de Odoo, activa el modo desarrollador, Aplicaciones -> Actualizar lista de aplicaciones -> Buscar módulo -> Instalar/Actualizar.

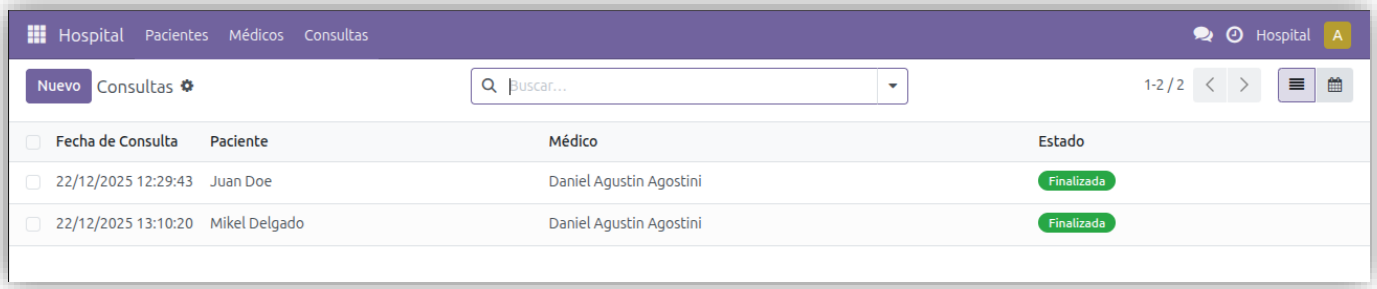
```
miriam@servidor-miriam:~/hospital/desarrollo$ docker restart desarrollo-web-1
desarrollo-web-1
miriam@servidor-miriam:~/hospital/desarrollo$ cd volumesOdoo/addons/hospital_gestion/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ ls
__init__.py  __manifest__.py  models  __pycache__  security  views
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ cd models/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/models$ ls
__init__.py  models.py  __pycache__
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/models$ nano __init__.py
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/models$ nano models.py
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/models$ cd ..
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ ls
__init__.py  __manifest__.py  models  __pycache__  security  views
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ cd security/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/security$ ls
ir.model.access.csv
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/security$ nano ir.model.access.csv
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/security$ cd ..
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ cd views/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$ nano views.xml
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$ cd ..
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ cd models/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/models$ nano models.py
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/models$ cd ..
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion$ cd views/
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$ nano views.xml
miriam@servidor-miriam:~/hospital/desarrollo/volumesOdoo/addons/hospital_gestion/views$
```

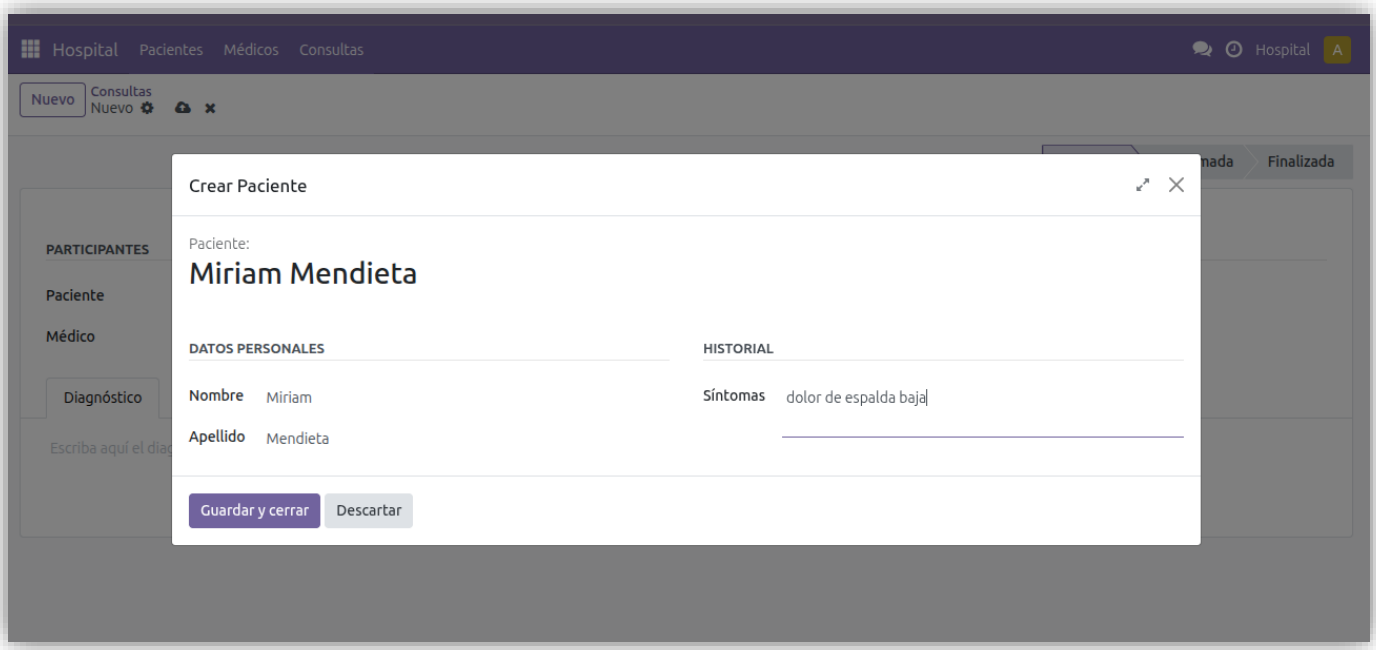


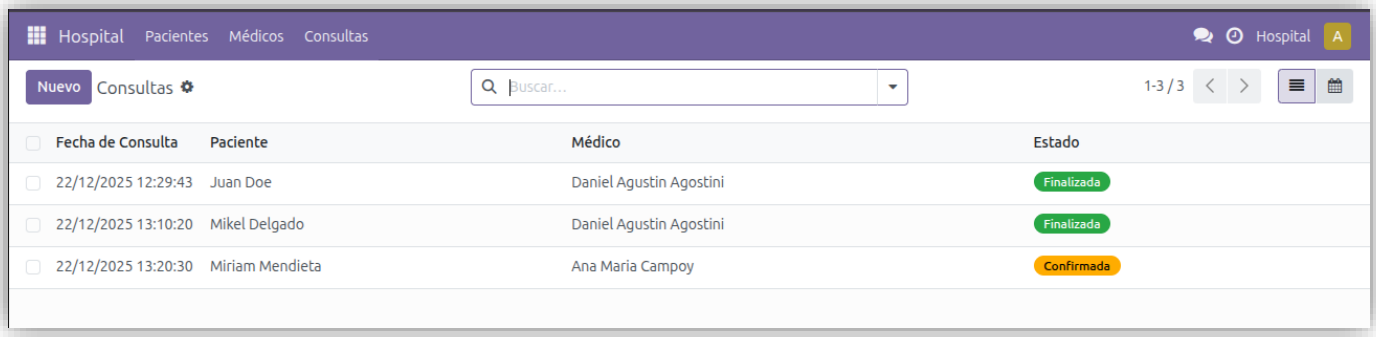

Hospital
Pacientes
Médicos
Consultas
Hospital
A
Nuevo
Pacientes
Buscar...
1-2 / 2
Nombre Completo
Síntomas
Juan Doe
Fiebre
Maria Becerra
dolor de barriga

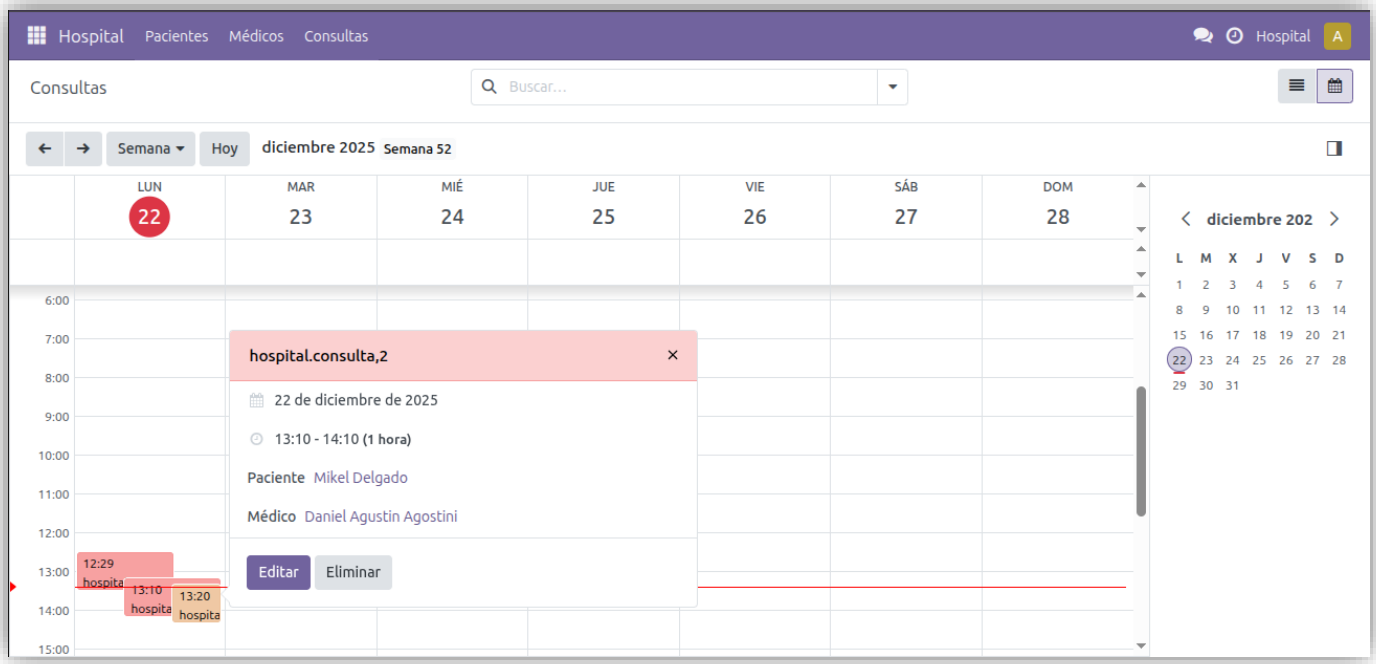

Hospital
Pacientes
Médicos
Consultas
Hospital
A
Nuevo
Pacientes
Mikel Delgado
1 / 1
Paciente:
Mikel Delgado
DATOS PERSONALES
Nombre Mikel
Apellido Delgado
HISTORIAL
Síntomas Dolor de cabeza


Hospital
Pacientes
Médicos
Consultas
Hospital
A
Nuevo
Médicos
Buscar...
1-2 / 2
Nombre Completo
Número de Colegiado
Daniel Agustin Agostini
2878231111
Ana Maria Campoy
4528111111
Ana Maria Campoy


Hospital
Pacientes
Médicos
Consultas
Hospital
A
Nuevo
Consultas
Buscar...
1-2 / 2
Fecha de Consulta
Paciente
Médico
Estado
22/12/2025 12:29:43
Juan Doe
Daniel Agustin Agostini
Finalizada
22/12/2025 13:10:20
Mikel Delgado
Daniel Agustin Agostini
Finalizada


Hospital
Pacientes
Médicos
Consultas
Nuevo
Consultas
Nuevo
Crear Paciente
Paciente:
Miriam Mendieta
DATOS PERSONALES
Nombre Miriam
Apellido Mendieta
HISTORIAL
Síntomas dolor de espalda baja
Guardar y cerrar
Descartar
PARTICIPANTES
Paciente
Médico
Diagnóstico
Finalizada


Hospital
Pacientes
Médicos
Consultas
Nuevo
Consultas
Buscar...
1-3 / 3
Fecha de Consulta
Paciente
Médico
Estado
22/12/2025 12:29:43
Juan Doe
Daniel Agustin Agostini
Finalizada
22/12/2025 13:10:20
Mikel Delgado
Daniel Agustin Agostini
Finalizada
22/12/2025 13:20:30
Miriam Mendieta
Ana Maria Campoy
Confirmada


Hospital
Pacientes
Médicos
Consultas
Consultas
Buscar...
Semana
Hoy
diciembre 2025
Semana 52
LUN 22
MAR 23
MIÉ 24
JUE 25
VIE 26
SÁB 27
DOM 28
hospital.consulta,2
22 de diciembre de 2025
13:10 - 14:10 (1 hora)
Paciente Mikel Delgado
Médico Daniel Agustin Agostini
Editar
Eliminar
diciembre 202

ACTIVIDAD 04

Haz un módulo de Odoo que represente nuestros estudios de ciclos formativos en un instituto:

- Modelo Ciclo formativo. Cada instancia representa un ciclo formativo en un instituto.
- Un ciclo tiene uno o varios módulos asociados
- Modelo Módulo. Cada "módulo" estará relacionado con ciclos formativos (al que pertenece), alumnos matriculados y profesor que lo imparte
- Modelo Alumno. Relacionado con los módulos matriculados.
- Modelo Profesor. Relacionado con los módulos que imparte.

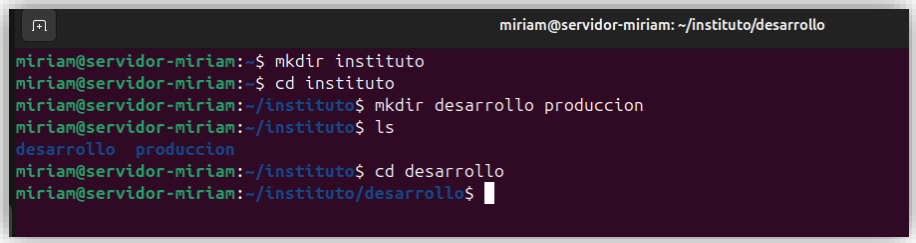
Implementa los modelos, las relaciones necesarias y las vistas que creas adecuadas para los 4 modelos.

Una vez en funcionamiento la aplicación, queremos que implementes la siguiente configuración de seguridad:

- Los usuarios con el rol "Director" podrán modificar los registros de los modelos anteriores.
- Además del director, los únicos usuarios que podrán ver los datos de los profesores (en modo lectura) serán los usuarios con rol "Profesor".

Estructura inicial del proyecto

```
instituto/
├──── desarrollo/
│     ├──── docker-compose.yml
└──── produccion/
```



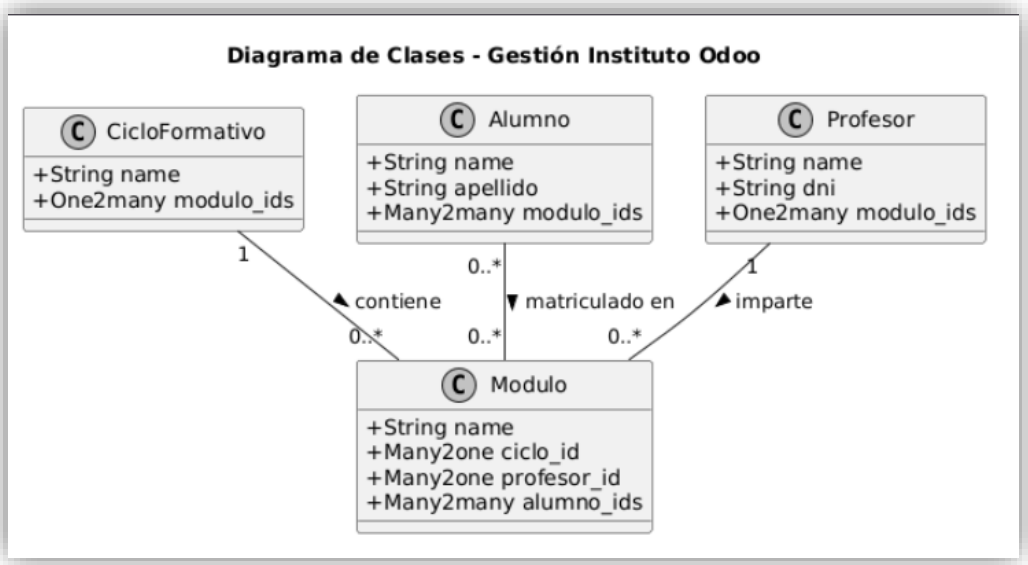

```
miriam@servidor-miriam:~/instituto$ cd desarrollo
miriam@servidor-miriam:~/instituto/desarrollo$ cp ~/biblioteca/desarrollo/docker-compose.yml ~/instituto/desarrollo/
```

1. Creación Estructura de Archivos del módulo instituto_gestion (scaffold)

```
Instituto_gestion/
├── __init__.py
├── __manifest__.py
├── models/
│   ├── __init__.py
│   └── models.py
├── security/
│   ├── groups.xml
│   └── ir.model.access.csv
└── views/
    └── views.xml
```

```
miriam@servidor-miriam:~/instituto/desarrollo/volumesOdoo/addons/instituto_gestion
miriam@servidor-miriam:~/instituto$ cd desarrollo
miriam@servidor-miriam:~/instituto/desarrollo$ docker compose exec -u root web odoo scaffold instituto_gestion /mnt/extra-addons
WARN[0000] /home/miriam/instituto/desarrollo/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please
 remove it to avoid potential confusion
miriam@servidor-miriam:~/instituto/desarrollo$ cd volumesOdoo/addons/instituto_gestion/
miriam@servidor-miriam:~/instituto/desarrollo/volumesOdoo/addons/instituto_gestion$ tree
.
├── controllers
│   ├── controllers.py
│   └── __init__.py
├── demo
│   └── demo.xml
├── __init__.py
├── __manifest__.py
├── models
│   ├── __init__.py
│   └── models.py
├── security
│   └── ir.model.access.csv
└── views
    ├── templates.xml
    └── views.xml
```

2. https://plantuml.com/es/



3. **Otorgar permisos**, borrar carpetas generadas por defecto, modificar init y manifest. #__manifest__.py

```
# -*- coding: utf-8 -*-
{
    'name': "Gestión de Instituto",

    'summary': """
        Módulo para la gestión de ciclos formativos, módulos, alumnos y profesores.""",

    'description': """
        Este módulo permite:
        - Administrar Ciclos Formativos.
        - Gestionar Módulos y sus asignaciones.
        - Matricular Alumnos en diferentes módulos.
        - Asignar Profesores a sus respectivos módulos.
        - Configuración de seguridad por roles (Director y Profesor).
    """,

    'author': "Miriam Mendieta",
    'website': "https://www.tuinstituto.com",

    # Categoría para organizar el módulo en el listado de Odoo
    'category': 'Education',
    'version': '0.1',

    # 'base' es el núcleo necesario para cualquier módulo.
    'depends': ['base'],

    # cargar los archivos de seguridad antes que las vistas
    'data': [
        'security/groups.xml',
        'security/ir.model.access.csv',
        'views/instituto_views.xml',
    ],
    'installable': True,
    'application': True,
    'auto_install': False,
    'license': 'LGPL-3',
}
```

4. Definición de Modelos (models/models.py)

Se definen la estructura de los datos y campos relacionales para conectar las entidades.

```
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class CicloFormativo(models.Model):
    _name = 'instituto.ciclo'
    _description = 'Ciclo Formativo'

    name = fields.Char(string='Nombre del Ciclo', required=True)
    modulo_ids = fields.One2many('instituto.modulo', 'ciclo_id', string='Módulos')

    _sql_constraints = [
        ('name_unique', 'unique(name)', '¡Ya existe un ciclo con este nombre!')
    ]

class Modulo(models.Model):
    _name = 'instituto.modulo'
    _description = 'Módulo Formativo'
    _order = 'name'

    name = fields.Char(string='Nombre del Módulo', required=True)
    ciclo_id = fields.Many2one('instituto.ciclo', string='Ciclo Formativo', ondelete='cascade')
    profesor_id = fields.Many2one('instituto.profesor', string='Profesor a Cargo')
    alumno_ids = fields.Many2many('instituto.alumno', string='Alumnos Matriculados')

    horas_totales = fields.Integer(string='Horas Totales', default=100)

    state = fields.Selection([
        ('draft', 'Borrador'),
        ('active', 'En Curso'),
        ('closed', 'Finalizado')
    ], string='Estado', default='draft')

    _sql_constraints = [
        ('name_ciclo_unique', 'unique(name, ciclo_id)', '¡Este módulo ya existe en este ciclo!')
    ]

    @api.constrains('horas_totales')
    def _check_horas(self):
        for record in self:
            if record.horas_totales <= 0:
                raise ValidationError("Las horas totales deben ser un número positivo.")
```

```
class Alumno(models.Model):
    _name = 'instituto.alumno'
    _description = 'Alumno'
    _rec_name = 'nombre_completo'

    photo = fields.Binary(string="Foto")
    name = fields.Char(string='Nombre', required=True)
    apellido = fields.Char(string='Apellidos')
    photo = fields.Binary(string="Foto")
    nombre_completo = fields.Char(string='Nombre Completo', compute='_compute_nombre_completo',
store=True)
    nia = fields.Char(string='NIA', required=True)
    modulo_ids = fields.Many2many('instituto.modulo', string='Módulos Inscritos')

    _sql_constraints = [
        ('nia_unique', 'unique(nia)', '¡El NIA ya existe!')
    ]

    @api.depends('name', 'apellido')
    def _compute_nombre_completo(self):
        for record in self:
            partes = [n for n in [record.name, record.apellido] if n]
            record.nombre_completo = " ".join(partes) if partes else ""

class Profesor(models.Model):
    _name = 'instituto.profesor'
    _description = 'Profesor'
    _rec_name = 'nombre_completo'

    photo = fields.Binary(string="Foto")
    name = fields.Char(string='Nombre', required=True)
    apellido = fields.Char(string='Apellidos')
    photo = fields.Binary(string="Foto")
    nombre_completo = fields.Char(string='Nombre Completo',compute='_compute_nombre_completo',
            store=True)
    dni = fields.Char(string='DNI', required=True)
    modulo_ids = fields.One2many('instituto.modulo', 'profesor_id', string='Módulos que imparte')

    _sql_constraints = [
        ('dni_unique', 'unique(dni)', '¡El DNI ya existe!')
    ]

    @api.depends('name', 'apellido')
    def _compute_nombre_completo(self):
        for record in self:
            partes = [n for n in [record.name, record.apellido] if n]
            record.nombre_completo = " ".join(partes) if partes else ""
```

Relaciones establecidas:

Modelo	Relación Principal	Tipo de Relación
Ciclo	Hacia Módulos	One2many (Un ciclo, muchos módulos)
Módulo	Hacia Ciclo	Many2one (Muchos módulos pertenecen a un ciclo)
Módulo	Hacia Alumnos	Many2many (Muchos alumnos en muchos módulos)
Módulo	Hacia Profesor	Many2one (Un profesor imparte el módulo)
Profesor	Hacia Módulos	One2many (Un profesor tiene varios módulos)

5. Definición de Grupos (security/groups.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
    <record id="module_category_instituto" model="ir.module.category">
        <field name="name">Gestión Instituto</field>
        <field name="description">Nivel de acceso para el sistema del instituto</field>
    </record>

    <record id="group_profesor" model="res.groups">
        <field name="name">Profesor</field>
        <field name="category_id" ref="module_category_instituto"/>
        <field name="users" eval="[(4, ref('base.user_root')), (4, ref('base.user_admin'))]"/>
    </record>

    <record id="group_director" model="res.groups">
        <field name="name">Director</field>
        <field name="category_id" ref="module_category_instituto"/>
        <field name="implied_ids" eval="[(4, ref('group_profesor'))]"/>
    </record>
</odoo>
```

6. Reglas de Acceso (security/ir.model.access.csv)

Se otorga permisos de Lectura (read), Escritura (write), Creación (create) y Eliminación.

- El Director tiene 1,1,1,1 en todos los modelos.
- El Profesor tiene 1,0,0,0 (solo lectura) en el modelo de profesores, y permisos completos en módulos y alumnos para que pueda trabajar en ellos.

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_ciclo_director,ciclo.director,model_instituto_ciclo,instituto_gestion.group_director,1,1,1,1
access_modulo_director,modulo.director,model_instituto_modulo,instituto_gestion.group_director,1,1,1,1
access_alumno_director,alumno.director,model_instituto_alumno,instituto_gestion.group_director,1,1,1,1
access_profesor_director,profesor.director,model_instituto_profesor,instituto_gestion.group_director,1,1,1,1
access_profesor_prof,profesor.prof,model_instituto_profesor,instituto_gestion.group_profesor,1,0,0,0
access_modulo_prof,modulo.prof,model_instituto_modulo,instituto_gestion.group_profesor,1,1,0,0
access_alumno_prof,alumno.prof,model_instituto_alumno,instituto_gestion.group_profesor,1,1,0,0
access_ciclo_prof,ciclo.prof,model_instituto_ciclo,instituto_gestion.group_profesor,1,0,0,0
access_ciclo_admin,access.ciclo.admin,model_instituto_ciclo,base.group_user,1,1,1,1
access_modulo_admin,access.modulo.admin,model_instituto_modulo,base.group_user,1,1,1,1
access_alumno_admin,access.alumno.admin,model_instituto_alumno,base.group_user,1,1,1,1
access_profesor_admin,access.profesor.admin,model_instituto_profesor,base.group_user,1,1,1,1
```

7. Vistas (views/views.xml)

Se define la interfaz de usuario organizada por menús y vistas(list, form, Kanban).

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <data>
   <record id="view_ciclo_search" model="ir.ui.view">
      <field name="name">instituto.ciclo.search</field>
      <field name="model">instituto.ciclo</field>
      <field name="arch" type="xml">
         <search>
            <field name="name"/>
         </search>
      </field>
   </record>

   <record id="view_ciclo_list" model="ir.ui.view">
      <field name="name">instituto.ciclo.list</field>
      <field name="model">instituto.ciclo</field>
      <field name="arch" type="xml">
         <list string="Ciclos">
            <field name="name"/>
         </list>
      </field>
   </record>

   <record id="view_ciclo_form" model="ir.ui.view">
      <field name="name">instituto.ciclo.form</field>
      <field name="model">instituto.ciclo</field>
      <field name="arch" type="xml">
         <form>
            <sheet>
               <div class="oe_title">
                  <label for="name"/>
                  <h1><field name="name"/></h1>
               </div>
               <notebook>
                  <page string="Módulos del Ciclo">
                     <field name="modulo_ids">
                        <list editable="bottom">
                           <field name="name"/>
                           <field name="profesor_id"/>
                           <field name="horas_totales"/>
                        </list>
                     </field>
                  </page>
               </notebook>
```

```
        </sheet>
      </form>
    </field>
</record>

<record id="action_ciclo" model="ir.actions.act_window">
    <field name="name">Ciclos</field>
    <field name="res_model">instituto.ciclo</field>
    <field name="view_mode">list,form</field>
</record>

<record id="view_modulo_search" model="ir.ui.view">
    <field name="name">instituto.modulo.search</field>
    <field name="model">instituto.modulo</field>
    <field name="arch" type="xml">
      <search>
        <field name="name"/>
        <field name="ciclo_id"/>
        <field name="profesor_id"/>
      </search>
    </field>
</record>

<record id="view_modulo_list" model="ir.ui.view">
    <field name="name">instituto.modulo.list</field>
    <field name="model">instituto.modulo</field>
    <field name="arch" type="xml">
      <list string="Módulos">
        <field name="name"/>
        <field name="ciclo_id"/>
        <field name="profesor_id" widget="many2one_avatar_user"/>
        <field name="horas_totales" sum="Total Horas"/>
      </list>
    </field>
</record>
```

```
<record id="view_modulo_kanban" model="ir.ui.view">
  <field name="name">instituto.modulo.kanban</field>
  <field name="model">instituto.modulo</field>
  <field name="arch" type="xml">
    <kanban>
      <templates>
        <t t-name="kanban-box">
          <div class="oe_kanban_global_click">
            <div class="oe_kanban_details">
              <strong class="o_kanban_record_title"><field name="name"/></strong>
              <div><field name="ciclo_id"/></div>
              <div><strong><field name="horas_totales"/> horas</strong></div>
              <div class="mt-2"><field name="profesor_id" widget="many2one_avatar_user"/></div>
            </div>
          </div>
        </t>
      </templates>
    </kanban>
  </field>
</record>

<record id="action_modulo" model="ir.actions.act_window">
  <field name="name">Módulos</field>
  <field name="res_model">instituto.modulo</field>
  <field name="view_mode">list,kanban,form</field>
</record>

<record id="view_alumno_search" model="ir.ui.view">
  <field name="name">instituto.alumno.search</field>
  <field name="model">instituto.alumno</field>
  <field name="arch" type="xml">
    <search>
      <field name="nombre_completo"/>
      <field name="nia"/>
    </search>
  </field>
</record>

<record id="view_alumno_list" model="ir.ui.view">
  <field name="name">instituto.alumno.list</field>
  <field name="model">instituto.alumno</field>
  <field name="arch" type="xml">
    <list string="Alumnos">
      <field name="nia"/>
      <field name="nombre_completo"/>
      <field name="modulo_ids" widget="many2many_tags"/>
    </list>
  </field>
</record>
```

```
<record id="view_alumno_kanban" model="ir.ui.view">
  <field name="name">instituto.alumno.kanban</field>
  <field name="model">instituto.alumno</field>
  <field name="arch" type="xml">
    <kanban>
      <field name="id"/>
      <field name="photo"/>
      <templates>
        <t t-name="kanban-box">
          <div class="oe_kanban_global_click o_kanban_record_has_image_fill">
            <div class="o_kanban_image_fill_left d-none d-md-block"
              t-attf-style="background-image:url('#{kanban_image('instituto.alumno', 'photo', re-
cord.id.raw_value)}')"/>
            <div class="oe_kanban_details">
              <strong class="o_kanban_record_title"><field name="nombre_completo"/></strong>
              <div><span>NIA: <field name="nia"/></span></div>
            </div>
          </div>
        </t>
      </templates>
    </kanban>
  </field>
</record>

<record id="view_alumno_form" model="ir.ui.view">
  <field name="name">instituto.alumno.form</field>
  <field name="model">instituto.alumno</field>
  <field name="arch" type="xml">
    <form>
      <sheet>
        <field name="photo" widget="image" class="oe_avatar" options="{'preview_image': 'photo'}"/>
        <div class="oe_title">
          <h1><field name="nombre_completo" readonly="1"/></h1>
        </div>
        <group>
          <group><field name="name"/><field name="apellido"/></group>
          <group><field name="nia"/></group>
        </group>
        <notebook>
          <page string="Matrícula">
            <field name="modulo_ids">
              <list>
                <field name="name"/>
                <field name="ciclo_id"/>
                <field name="horas_totales"/>
              </list>
            </field>
          </page>
        </notebook>
```

```
            </sheet>
        </form>
    </field>
</record>

<record id="action_alumno" model="ir.actions.act_window">
    <field name="name">Alumnos</field>
    <field name="res_model">instituto.alumno</field>
    <field name="view_mode">list,kanban,form</field>
</record>

<record id="view_profesor_search" model="ir.ui.view">
    <field name="name">instituto.profesor.search</field>
    <field name="model">instituto.profesor</field>
    <field name="arch" type="xml">
        <search>
            <field name="nombre_completo"/>
            <field name="dni"/>
        </search>
    </field>
</record>

<record id="view_profesor_list" model="ir.ui.view">
    <field name="name">instituto.profesor.list</field>
    <field name="model">instituto.profesor</field>
    <field name="arch" type="xml">
        <list>
            <field name="dni"/>
            <field name="nombre_completo"/>
        </list>
    </field>
</record>

<record id="view_profesor_kanban" model="ir.ui.view">
    <field name="name">instituto.profesor.kanban</field>
    <field name="model">instituto.profesor</field>
    <field name="arch" type="xml">
        <kanban>
            <field name="id"/>
            <field name="photo"/>
            <templates>
                <t t-name="kanban-box">
                    <div class="oe_kanban_global_click o_kanban_record_has_image_fill">
                        <div class="o_kanban_image_fill_left d-none d-md-block"
                            t-attf-style="background-image:url('#{kanban_image('instituto.profesor', 'photo', record.id.raw_va
                                lue)}')"/>
                        <div class="oe_kanban_details">
                            <strong class="o_kanban_record_title"><field name="nombre_completo"/></strong>
                            <div><span>DNI: <field name="dni"/></span></div>
                        </div>
                    </div>
```

```
            </t>
          </templates>
        </kanban>
      </field>
    </record>

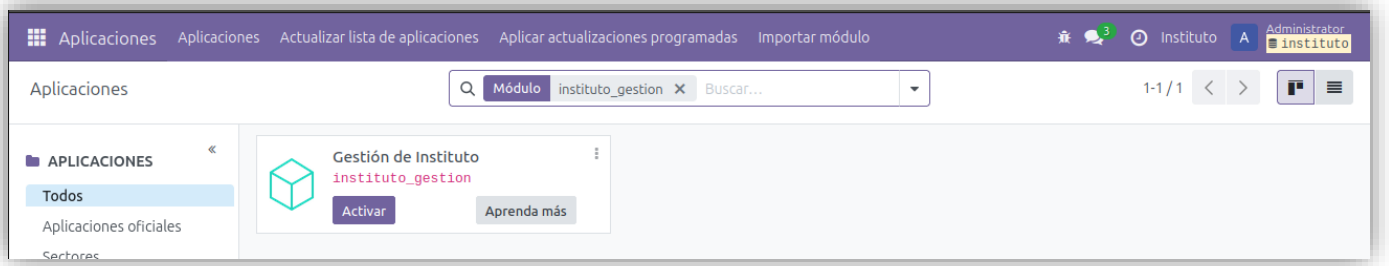
    <record id="view_profesor_form" model="ir.ui.view">
      <field name="name">instituto.profesor.form</field>
      <field name="model">instituto.profesor</field>
      <field name="arch" type="xml">
        <form>
          <sheet>
            <field name="photo" widget="image" class="oe_avatar"/>
            <div class="oe_title">
              <h1><field name="nombre_completo" readonly="1"/></h1>
            </div>
            <group>
              <group><field name="name"/><field name="apellido"/></group>
              <group><field name="dni"/></group>
            </group>
            <notebook>
              <page string="Módulos Impartidos">
                <field name="modulo_ids">
                  <list>
                    <field name="name"/>
                    <field name="horas_totales"/>
                  </list>
                </field>
              </page>
            </notebook>
          </sheet>
        </form>
      </field>
    </record>

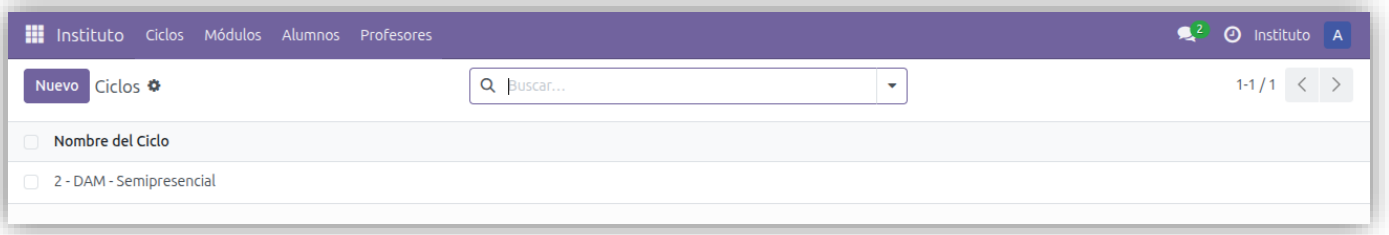
    <record id="action_profesor" model="ir.actions.act_window">
      <field name="name">Profesores</field>
      <field name="res_model">instituto.profesor</field>
      <field name="view_mode">list,kanban,form</field>
    </record>

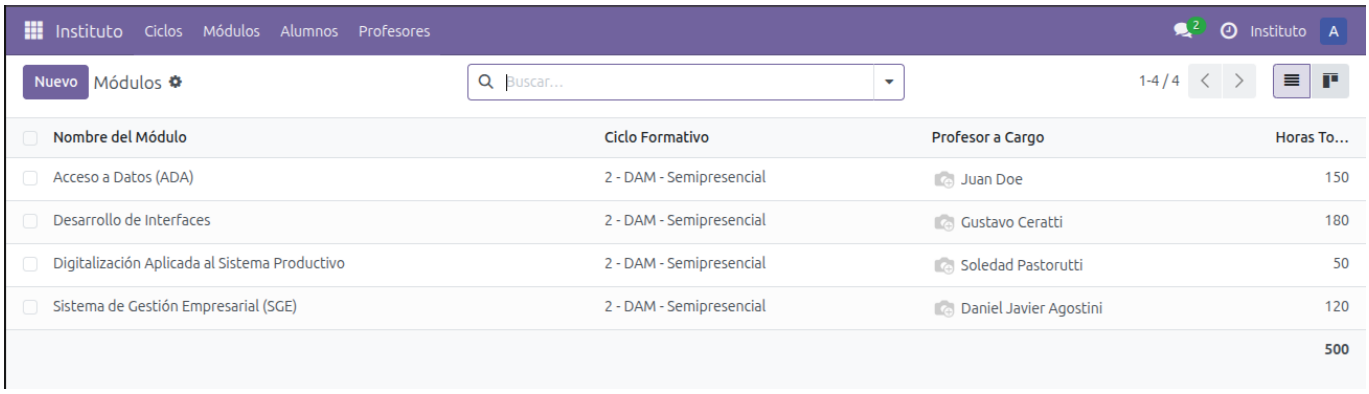
    <menuitem id="menu_root" name="Instituto" sequence="10"/>
    <menuitem id="menu_ciclo" name="Ciclos" parent="menu_root" action="action_ciclo" sequence="10"/>
    <menuitem id="menu_modulo" name="Módulos" parent="menu_root" action="action_modulo" sequence="20"/>
    <menuitem id="menu_alumno" name="Alumnos" parent="menu_root" action="action_alumno" sequence="30"/>
    <menuitem id="menu_profesor" name="Profesores" parent="menu_root" action="action_profesor" sequence="40"/>
  </data>
</odoo>
```

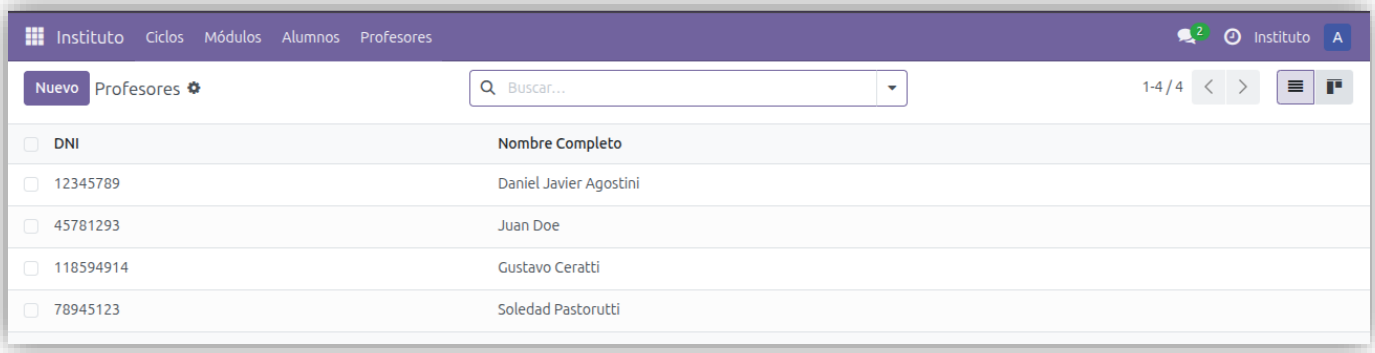
8. **Reiniciar el contenedor** y Activar el Modo Desarrollador para instalar el módulo

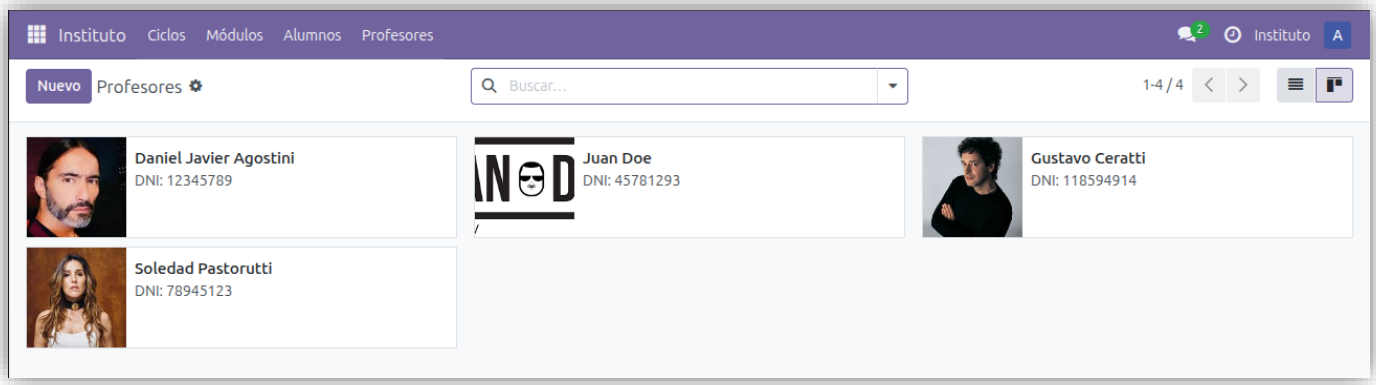


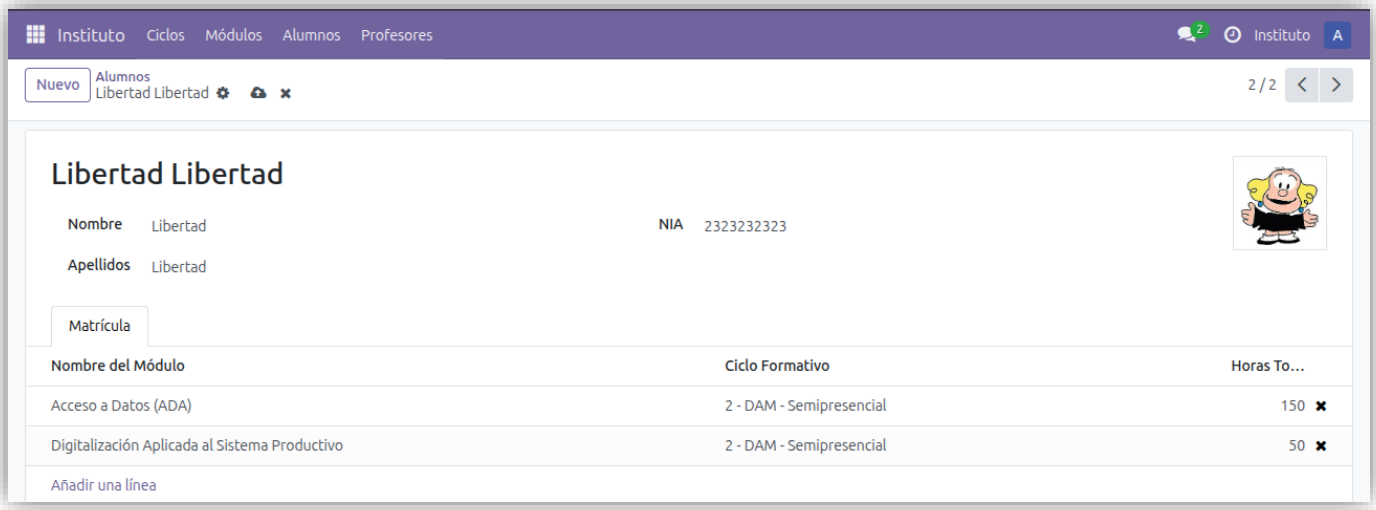


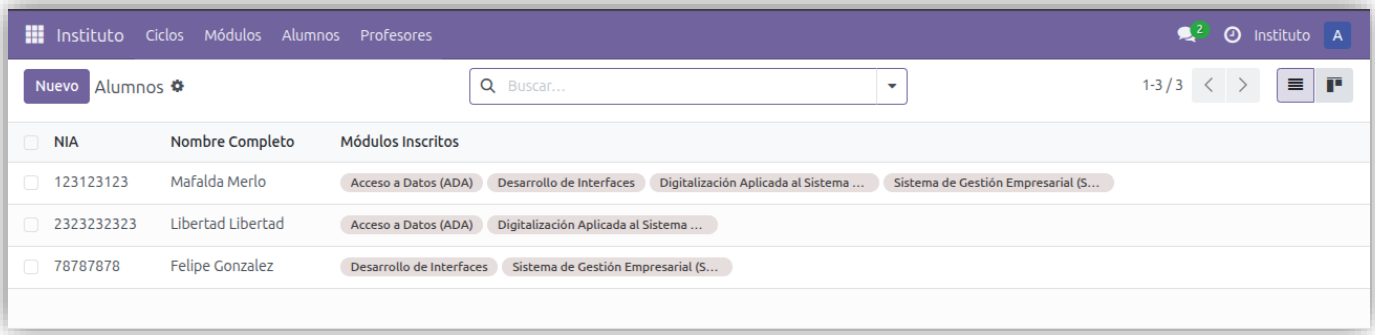


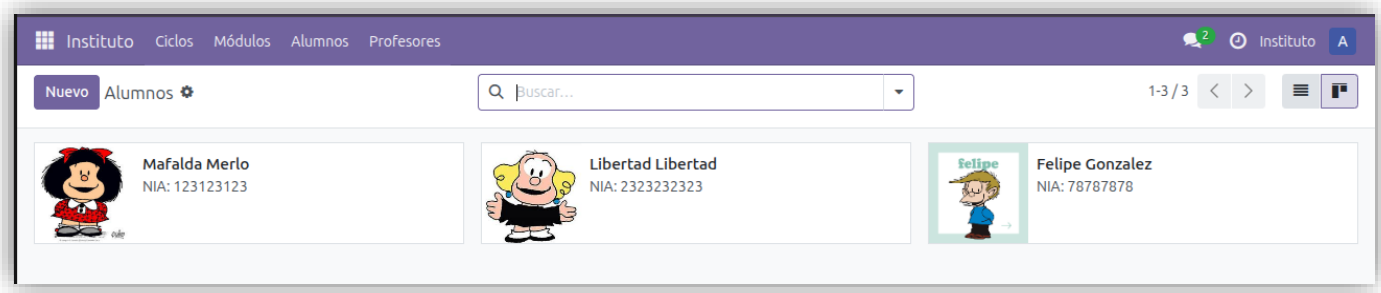



Instituto Ciclos Módulos Alumnos Profesores
Instituto A
Nuevo Profesores
Buscar...
1-4 / 4
DNI Nombre Completo
12345789 Daniel Javier Agostini
45781293 Juan Doe
118594914 Gustavo Ceratti
78945123 Soledad Pastorutti


Instituto Ciclos Módulos Alumnos Profesores
Instituto A
Nuevo Profesores
Buscar...
1-4 / 4
Daniel Javier Agostini
DNI: 12345789
Juan Doe
DNI: 45781293
Gustavo Ceratti
DNI: 118594914
Soledad Pastorutti
DNI: 78945123


Instituto Ciclos Módulos Alumnos Profesores
Instituto A
Nuevo Alumnos
Libertad Libertad
2 / 2
Libertad Libertad
Nombre Libertad
Apellidos Libertad
NIA 2323232323
Matrícula
Nombre del Módulo Ciclo Formativo Horas To...
Acceso a Datos (ADA) 2 - DAM - Semipresencial 150
Digitalización Aplicada al Sistema Productivo 2 - DAM - Semipresencial 50
Añadir una línea


Instituto Ciclos Módulos Alumnos Profesores
Instituto A
Nuevo Alumnos
Buscar...
1-3 / 3
NIA Nombre Completo Módulos Inscritos
123123123 Mafalda Merlo Acceso a Datos (ADA) Desarrollo de Interfaces Digitalización Aplicada al Sistema ... Sistema de Gestión Empresarial (S...
2323232323 Libertad Libertad Acceso a Datos (ADA) Digitalización Aplicada al Sistema ...
78787878 Felipe Gonzalez Desarrollo de Interfaces Sistema de Gestión Empresarial (S...



Conclusión

Estos proyectos demuestran que ***Odoo*** trasciende la definición de un ERP convencional; funciona como un potente motor de base de datos relacional con una capa de interfaz altamente flexible. La capacidad de orquestar relaciones complejas (One2many, Many2one, Many2many) y representar-las mediante diversos tipos de vistas lo consolida como una herramienta integral para la gestión de activos. Es preciso subrayar que una implementación exitosa de módulos personalizados no solo exige el conocimiento de Python y la sintaxis XML, sino también una comprensión profunda del flujo de datos entre la lógica del servidor y PostgreSQL. En conclusión, el orden jerárquico de carga en el manifiesto y la precisión en la definición de las reglas de acceso son los pilares fundamentales que garantizan la estabilidad y escalabilidad de cualquier solución empresarial desarrollada sobre este ecosistema.

BIBLIOGRAFIA

- Apuntes del Profesor.
- https://www.odoo.com/documentation/18.0/es/
- Python para todos. Explorando la información con Python 3. Charles R. Severance
- https://docs.python.org/es/3/library/copy.html
- https://www.plantuml.com/plantuml/uml/SyfFKj2rKt3CoKnELR1Io4ZDoSa700003#google_vignette
- https://www.youtube.com/watch?v=L75Fgs3XJMk&list=PL1-aSABtP6ABaEl9nfhPnb7Dss4Hv2ELS
- https://www.youtube.com/watch?v=5pcsjxbqmFw&list=PLRg_SI-2J2Yb1EZp5RVh4GMj-YnoqLDp1
- https://www.youtube.com/watch?v=bwYD9gXeh2E&list=PLXwzJRuH--eYHB3OD2wATjzJ83UAp2WQs
- https://www.plantuml.com/plantuml/uml/SyfFKj2rKt3CoKnELR1Io4ZDoSa700001

Credenciales de acceso

- URL de acceso: http://localhost:8069
- Usuario administrador: admin
- Contraseña: 1722
- Servidor: miriam /1722